

Development of a Multimodal System for Converting PDF Documents into Structured Markdown

Noor Martini

Bachelor Thesis

for the acquisition of the academic degree Bachelor of Science (B.Sc.)

Course of Studies: Enterprise Computing

Department of Computer Science
University of Applied Sciences Mannheim

2026-07-14

Tutors

Prof. Dr. Jörn Fischer, Hochschule Mannheim

Prof. Dr. Thomas Ihme

Martini, Noor:

Entwicklung eines multimodalen Systems zur Konvertierung von PDF-Dokumenten in strukturiertes Markdown / Noor Martini. –

Bachelor-Thesis, Mannheim: Technische Hochschule Mannheim, 2026. 88 Seiten.

Martini, Noor:

Development of a Multimodal System for Converting PDF Documents into Structured Markdown / Noor Martini. –

Bachelor Thesis, Mannheim: University of Applied Sciences Mannheim, 2026. 88 pages.

Erklärung / *Declaration*

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

I confirm that the submitted thesis is original work and was written by me without further assistance. Appropriate credit has been given where reference has been made to the work of others.

Mannheim, 2026-07-14



Noor Martini

I agree that my work may be published, i.e. that the work may be stored electronically, converted into other formats, made publicly available on the servers of the University of Applied Sciences Mannheim and distributed via the Internet.

Abstract

Development of a Multimodal System for Converting PDF Documents into Structured Markdown

PDF documents are the dominant format for sharing structured content across academia and professional domains, but they store pages as rendering instructions rather than semantic elements, making automated content extraction unreliable. This thesis designs, implements, and evaluates a multimodal PDF-to-Markdown conversion system that classifies each page individually into one of six content types (TEXT, IMAGE, FORMULA, TABLE, MIXED, or EMPTY) and routes each page to the most appropriate extraction strategy. Plain-text pages are processed by PyMuPDF without invoking a language model; visually complex pages are sent to a locally hosted vision-language model via LM Studio, eliminating any dependency on external cloud services. The system is evaluated on 90 pages from three bachelor thesis documents using a seven-metric framework covering text similarity, heading structure, list structure, table structure, code block accuracy, paragraph structure, and word overlap. The adaptive strategy beats both fixed vision-based strategies outright on every metric and is closely matched with direct text extraction, leading on list structure, table structure, and word overlap, tying on text similarity, and trailing narrowly on the remaining three. Its clearest advantage is table structure accuracy (0.978 versus 0.933 for text-only), and it reduces total token consumption by 75% compared to image-only processing. A two-stage post-processing pipeline removes LLM output artefacts and reconstructs document structure through 39 transformation passes, verified by 182 unit tests. The results show that per-page strategy selection matches or beats any uniform fixed strategy, at a fraction of the cost of the fixed vision strategies, while direct text extraction remains a fast, competitive choice for prose-heavy documents specifically.

Abstrakt

Entwicklung eines multimodalen Systems zur Konvertierung von PDF-Dokumenten in strukturiertes Markdown

PDF-Dokumente sind das dominierende Format für den Austausch strukturierter Inhalte in Wissenschaft und Praxis, speichern Seiten jedoch als Rendering-Anweisungen statt als

semantische Elemente, was eine automatisierte Inhaltsextraktion erschwert. Diese Bachelorarbeit entwirft, implementiert und evaluiert ein multimodales System zur Konvertierung von PDF-Dokumenten in strukturiertes Markdown, das jede Seite einzeln in einen von sechs Inhaltstypen klassifiziert (TEXT, IMAGE, FORMULA, TABLE, MIXED oder EMPTY) und an die jeweils geeignetste Extraktionsstrategie weiterleitet. Reine Textseiten werden von PyMuPDF ohne LLM-Aufruf verarbeitet; visuell komplexe Seiten werden an ein lokal gehostetes Vision-Language-Modell über LM Studio übergeben, ohne Abhängigkeit von externen Cloud-Diensten. Das System wird auf 90 Seiten aus drei Bachelorarbeiten mit einem Sieben-Metrik-Rahmen evaluiert, der Textähnlichkeit, Überschriftenstruktur, Listenstruktur, Tabellenstruktur, Codeblock-Genauigkeit, Absatzstruktur und Wortüberschneidung abdeckt. Die adaptive Strategie schlägt beide bildbasierten Fixstrategien in jeder einzelnen Metrik und liegt insgesamt gleichauf mit der reinen Textextraktion: Sie führt bei Listenstruktur, Tabellenstruktur und Wortüberschneidung, liegt bei der Textähnlichkeit gleichauf und bei den übrigen drei Metriken nur knapp zurück. Ihr klarster Vorteil liegt in der Tabellenstruktur (0,978 gegenüber 0,933 bei reiner Textextraktion), und sie reduziert den gesamten Token-Verbrauch um 75 % gegenüber der reinen Bildextraktion. Eine zweistufige Nachverarbeitungspipeline entfernt LLM-Ausgabe-Artefakte und rekonstruiert die Dokumentstruktur durch 39 Transformationsschritte, überprüft durch 182 Unit-Tests. Die Ergebnisse zeigen, dass die seitenweise Strategiewahl jede einheitliche Fixstrategie erreicht oder übertrifft, bei einem Bruchteil der Kosten der bildbasierten Fixstrategien, während die reine Textextraktion für textdominante Dokumente speziell weiterhin eine schnelle, konkurrenzfähige Wahl bleibt.

Contents

List of Abbreviations	ix
List of Figures	xi
List of Tables	xii
Listings	xiii
1 Introduction	1
1.1 Problem Statement	2
1.1.1 Difficulty of Extracting Structured Content from PDFs	2
1.1.2 Limited OCR Accuracy for Complex Layouts	2
1.1.3 Need for Conversion to Markdown for Accessibility and Version Control	3
1.2 Motivation	4
1.2.1 Growing Importance of Document Digitization	4
1.2.2 Advantages of Markdown as a Structured Representation	5
1.2.3 Comparison of Extraction Strategies	5
1.3 Research Objectives and Questions	6
1.4 Scope and Delimitations	7
1.5 Thesis Contributions	8
1.6 Thesis Structure	8
2 Background and Fundamentals	10
2.1 PDF Format and Parsing Challenges	10
2.2 Markdown Format	12
2.3 Large Language Models	13
2.4 Multimodal Processing Strategies	14
3 Related Work	16
3.1 Traditional PDF Processing Tools	16
3.1.1 Text-based Parsers	16

3.1.2	OCR Solutions	17
3.1.3	Shared Limitations	18
3.2	Modern Document Conversion Tools	18
3.3	LLM-based Document Understanding	19
3.4	Evaluation Methods for Document Conversion	20
4	Methodology	22
4.1	Research Design	22
4.2	Dataset and Ground Truth Preparation	23
4.3	Evaluation Metrics	24
4.4	Experimental Design	25
5	System Design and Implementation	27
5.1	Architecture Overview	27
5.2	Page Content Classifier	29
5.3	PDF Processing Module	31
5.4	LLM Integration	32
5.5	Post-processing Pipeline	33
5.6	Implementation Details	39
6	Results and Evaluation	41
6.1	Experiment Setup	41
6.2	Overall Strategy Performance (RQ1, RQ2)	42
6.3	Efficiency (RQ3)	44
6.4	Per-Page Analysis and Consistency (RQ5)	46
6.5	Output Quality Metrics (RQ4)	48
7	Discussion	50
7.1	Interpretation of Results	50
7.2	Comparison with Related Work	51
7.3	Practical Implications	52
7.4	Threats to Validity	53
8	Conclusion	56
8.1	Summary of Findings	56
8.2	Contributions and Impact	57
8.3	Limitations of the Study	59
8.4	Future Work	60

Bibliography	xiv
A Supplementary Material	xvii
A.1 Implementation Code Snippets	xvii
A.1.1 Page Classification Rules	xvii
A.1.2 Post-processing Pass Sequence	xix
A.1.3 Evaluation Metric Examples	xx
A.2 Complete Results Tables	xxii
A.3 Sample Documents and Outputs	xxii
A.4 Prompt Templates	xxii

List of Abbreviations

API	Application Programming Interface
BLEU	Bilingual Evaluation Understudy
DPI	Dots Per Inch
GFM	GitHub Flavored Markdown
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
LSTM	Long Short-Term Memory
mAP	mean Average Precision
OCR	Optical Character Recognition
PDF	Portable Document Format
PNG	Portable Network Graphics
RAG	Retrieval-Augmented Generation
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
TEDS	Tree Edit Distance based Similarity
URI	Uniform Resource Identifier

List of Abbreviations

VLM Vision-Language Model

VRD-CU Visually Rich Document Content Understanding

List of Figures

2.1	Sequential text extraction versus intended structure, shown on the first body page of one evaluation document. The raw extraction keeps the visual line breaks, including mid-word hyphenation, and carries no heading markup.	11
4.1	Evaluation pipeline: corpus, strategies, ground truth, metrics, mean scores, and the paired permutation test	22
5.1	System architecture: from PDF input to Markdown output	29
5.2	Caption reordering on a corpus page (sentiment analysis thesis, page 9, classified as MIXED). The vision model emits the caption above the figure link. The pass swaps the two lines and removes the blank line, so the caption sits directly below its image. The link path is where <code>extract_page_figures</code> saved the embedded figure.	35
5.3	Source-code reconstruction on a corpus page (neural networks thesis, page 21). <code>pymupdf4llm</code> emits the listing as list items that merge two code lines each; the monospace-font pass rebuilds it as a fenced code block and leaves the caption line untouched.	36
5.4	Page classifier, phase 1: the classification cascade	38
5.5	Page classifier, phase 2: the routing checks	38
5.6	Two-stage post-processing pipeline	39
6.1	Mean metric scores per strategy across 90 pages	42
6.2	Mean processing time per page by strategy (log scale)	44
6.3	The same page under two prompts: HITL thesis, page 9 (List of Tables), converted twice by the same vision model from the same page image. Only the system prompt differs. Each output is shown after post-processing, truncated to the first three rows.	47

List of Tables

- 2.1 Common Markdown syntax elements 12
- 3.1 Comparison of PDF processing tools across dimensions relevant to structured document conversion 20
- 4.1 Comparison of PDF, Word (.docx), and Markdown as document formats . 23
- 4.2 Preliminary comparison of PDF extraction strategies 26
- 6.1 Mean metric scores per strategy across 90 pages (3 documents) 42
- 6.2 Paired sign-flip permutation tests: adaptive versus each fixed strategy over 90 pages. Δ is the mean per-page score difference (adaptive minus baseline); positive values favour adaptive. Bold marks $p < 0.05$ 43
- 6.3 Mean processing time per page by strategy 44
- 6.4 Token usage per strategy across 90 pages 45
- 6.5 Adaptive versus text-only per document: mean table structure and text similarity scores (30 pages per document) 48
- A.1 Mean metric scores and processing time per strategy (90 pages, $n = 90$) . xxii

Listings

- A.1 Page classification rules (strategies/adaptive.py) xvii
- A.2 Per-page post-processing pass sequence (postprocess.py) xix
- A.3 Text similarity and heading structure metrics (evaluation/metrics.py) xx

Chapter 1

Introduction

Portable Document Format (PDF) is the dominant format for sharing and archiving documents across law, finance, healthcare, academia and government. This is largely because a PDF looks identical on every device and cannot be accidentally edited. But that same property creates a serious problem for automated processing. PDF stores text as individual characters placed at specific coordinates on a page. There is no native concept of paragraphs, headings, or lists. Any tool that tries to extract content from a PDF has to reconstruct that structure from scratch, using only positional and typographic signals, which is unreliable [1].

Three main approaches exist for PDF-to-Markdown conversion. Text-based parsers such as PyMuPDF and pdfplumber [2] read the raw text layer of a native PDF and recover the characters, but discard layout structure in the process. Optical Character Recognition (OCR) tools such as Tesseract [3] treat the page as a flat image and recognize characters pixel by pixel. This works for scanned documents, but the output is plain text with no structure. A newer generation of AI-based systems, including Marker, Microsoft MarkIt-Down, LlamaParse, and multimodal models such as Mistral OCR [4], uses deep learning to recover more structure and handles tables, formulas, and multi-column layouts more reliably. Many pipelines combine all three stages: scanned pages go through OCR first, then a layout-aware conversion tool, then regex-based post-processing.

But all of these share one problem: they apply the same strategy to every page of a document. A tool designed for text-heavy papers uses that same approach on a page that contains only a scanned diagram or a dense formula. Real-world documents mix plain text, tables, images, source code, and equations, sometimes on the same page, and no single strategy handles all of them well. Ingemarsson and Persson [5] tested this directly: rule-based tools scored above 86% on plain-text PDFs but dropped to 0% on image-only pages. No single method came out on top across all content types. And to the best of the author's knowledge, no widely used open-source tool picks a different strategy per page based on what that page actually contains.

This thesis builds and evaluates a system that does exactly that. Instead of applying one fixed strategy to the whole document, the system classifies each page as plain text, image-heavy, formula-heavy, table-heavy, mixed content, or empty, and routes it to the extraction method best suited to that type. The system runs locally using LM Studio as the Large Language Model (LLM) runtime, so there is no dependency on external cloud services and results are fully reproducible. Converting a PDF to Markdown is more than a format change — it means recovering the document’s structure: headings, lists, tables, links, code blocks, and hierarchy, from a format that encodes visual appearance but not meaning. This chapter explains the problem, defines the research questions, and outlines the scope of the work.

1.1 Problem Statement

The problem this thesis addresses has two parts: the PDF format itself hides structural information, and existing tools cannot adjust their extraction approach based on what a given page actually contains.

1.1.1 Difficulty of Extracting Structured Content from PDFs

PDF was not designed to store meaning. It was designed to reproduce the visual appearance of a printed page. The format has no native representation of paragraphs, headings, list items, or table cells [1]. Any structure a reader sees has to be reconstructed by software from low-level positional and typographic signals, which is fragile and often wrong [6]. A single PDF can mix plain text, embedded images, mathematical formulas, multi-column layouts, and complex tables, sometimes on the same page. No single extraction strategy handles all of these correctly. Meuschke et al. [2] evaluated mainstream parsing tools across multiple document domains and found that even the top performers failed on scientific and patent documents.

1.1.2 Limited OCR Accuracy for Complex Layouts

OCR is the standard approach for scanned PDFs, where text exists only as pixels. But OCR accuracy drops significantly on complex layouts. Subramani et al. [7] describe OCR as a two-stage process: text detection, which locates where text appears on the page, and text transcription, which recognises characters within each detected region. Layout analysis must happen before both stages: the tool first needs to identify content regions (text

blocks, figures, tables, equations) before any recognition can begin. Blecher et al. [8] and Subramani et al. [7] explain the mechanical limitation: traditional OCR engines are built on a detect-first, recognise-second pattern where any error in layout segmentation propagates directly into text recognition, whereas modern unified visual models generate structured markup from full-page screenshots without a separate layout step. Tools like Tesseract [3] skip this step and treat the whole page as a single flat image, which makes it hard to assign the right semantic role to each detected region. Zhu et al. [6] measured the effect directly: without layout segmentation, normalized edit distance was 0.63; with a layout segmentation step added beforehand, it dropped to 0.38 (lower is better). That is a large gain from one preprocessing step alone.

Multi-column layouts are a major source of errors. OCR engines often merge text across columns or get the reading order wrong, and Zhu et al. [6] show that preserving correct reading order in multi-column documents requires a dedicated segmentation stage before any character recognition is attempted. Even current models struggle with dense multi-column text, rotated text, tables, and mathematical notation [2, 7]. Blecher et al. [8] note that the PDF format causes “a loss of semantic information, particularly for mathematical expressions,” and Darvishy [9] points out that no existing tool can interpret or describe mathematical formulas and scientific graphs. The core issue is structural, not just visual: what a page contains determines which extraction approach has any chance of working.

1.1.3 Need for Conversion to Markdown for Accessibility and Version Control

Markdown is a lightweight markup language created by John Gruber in 2004. It is readable as plain text and converts cleanly to HTML and other formats [10].

Converting PDFs to Markdown matters for accessibility. Darvishy [9] explains the problem: PDF stores pages as rendering instructions, not as structured content. Screen readers and other assistive technologies need semantic tags and a defined reading order to process a document correctly. Tagged PDF addresses this, but most PDFs in practice are untagged, and Darvishy found that even when tags are present, correct reading order and structure still require manual verification. For scientific documents the gap is larger: no existing PDF accessibility tool can interpret mathematical formulas or scientific graphs, which leaves that content unreachable for screen reader users [9]. Markdown marks structure directly in the text: headings with #, lists with -, formulas with \$. Assistive tools can process it without guessing. The GitHub Flavored Markdown extension adds table syntax and $\text{\LaTeX}$ -style math notation, which covers most of what academic documents require [11].

Markdown’s other advantages follow from being plain text. It works with version control systems like Git, where every change is tracked at the character level, which PDF and binary formats cannot offer. It is easy to parse programmatically and stays readable without any specific software. Khan et al. [12] show a concrete consequence for AI pipelines: the quality of text extraction sets a ceiling on retrieval precision in RAG systems. Poorly structured text produces low-quality embeddings, and retrieval fails even when the relevant content is present. Markdown’s structure prevents that degradation.

1.2 Motivation

This work is motivated by a clear gap: organizations increasingly need to process PDFs programmatically, but the tools available today either fail on complex pages or apply the same strategy everywhere regardless of whether it fits.

1.2.1 Growing Importance of Document Digitization

Document digitization is accelerating across all sectors. Organizations are not just scanning documents anymore. They need to extract, query, and transform document content as part of larger automated workflows. This is driven partly by the growth of AI-based systems that consume structured text, such as LLM-based assistants and RAG pipelines, where the quality of the input directly affects the quality of the output [12]. PDF-to-Markdown conversion is a key step in this pipeline: it bridges the gap between static archived documents and the structured, searchable knowledge bases that modern AI workflows depend on.

Scientific knowledge is a clear example. Blecher et al. [8] note that PDFs make up 2.4% of Common Crawl and are the second most common data format on the internet after HTML. Most of that content is not directly machine-readable. For AI systems that need to reason over large document collections, this is a serious bottleneck. In the RAG experiments of Khan et al. [12], the extraction step, not the language model, was the limiting factor.

The problem is not limited to academic papers. Ding et al. [13] identify finance, healthcare and legal domains as sectors where visually rich documents are the primary carrier of information. In all three, automated workflows increasingly need to read, compare and summarise document content at scale. PDF was not designed for this. It was designed for printing. Closing that gap between the printed format and machine-readable output is what motivates the work in this thesis.

1.2.2 Advantages of Markdown as a Structured Representation

Among common output formats, Markdown is the most practical choice for the use cases this thesis targets. It is human-readable without a renderer, compatible with Git for version control, easy to parse programmatically, and well-supported by LLM and RAG tooling.

The choice matters for downstream tasks. Markdown preserves hierarchy through headings, lists and tables, which means the document’s structure is visible to the embedding model and to any downstream reader. A plain paragraph dump does not provide that.

Markdown also has a practical advantage over XML and HTML for this setting. Both of those formats require a parser to read, and neither is easy to inspect by hand. Markdown is plain text. A developer can open the output file in any text editor and immediately see whether the conversion was correct. For a system that will be evaluated by comparing its output to hand-written reference files, this readability matters: it makes the reference creation process faster and the error analysis more direct.

1.2.3 Comparison of Extraction Strategies

The four strategies evaluated in this thesis differ in how they handle different content types. Text-only extraction is fast and does not require an LLM, but it cannot process scanned pages or pages dominated by images or formulas. Image-only extraction sends every page through a vision LLM, which handles all content types but applies that cost even to plain-text pages that do not need it. The hybrid strategy improves structural fidelity by combining both text and image signals, but still applies the same pipeline to every page uniformly. The adaptive approach routes each page to the strategy that fits its detected content type, which is the key difference none of the other three provide.

Meuschke et al.’s finding that no single tool performs consistently well across document types [2] directly motivates the adaptive approach: if the best tool for one document type is the wrong tool for another, a system that applies one strategy uniformly will always underperform on at least some content.

Ingemarsson and Persson [5] showed the cost side of the same trade-off: multimodal tools scored highest overall but applied the same high computation to every page regardless of content. The right extraction method depends on what a specific page contains, not on the document as a whole. Per-page adaptive routing is a direct answer to that observation.

1.3 Research Objectives and Questions

The main goal of this thesis is to design, implement, and evaluate a multimodal, content-aware system that converts PDF documents into structured Markdown by selecting an extraction strategy per page based on detected content type. The system uses LM Studio as a local LLM runtime to ensure full reproducibility without dependence on external cloud services. The central question is whether this per-page selection outperforms any single fixed strategy on documents with diverse content types, while reducing computational cost by skipping the LLM on plain-text pages.

In practice, plain-text pages are routed to PyMuPDF and visually complex pages are sent to a locally hosted vision LLM via LM Studio. A single document can use both pathways, depending on what each page contains.

This leads to five research questions.

RQ1 — Adaptivity. On documents with diverse content types, does per-page strategy selection achieve higher accuracy and structural fidelity than any single fixed strategy applied uniformly to the full document?

RQ2 — Strategy comparison. How do text-only, image-only, and hybrid strategies compare in terms of content and structural accuracy across different thesis documents?

RQ3 — Efficiency. What are the computational costs (processing time, token usage, and LLM call count) of each strategy and of the adaptive system, and how do those costs relate to accuracy?

RQ4 — Output quality. How can Markdown quality be reliably measured beyond BLEU (Bilingual Evaluation Understudy) and ROUGE (Recall-Oriented Understudy for Gisting Evaluation), given that structured output requires evaluating heading hierarchy, list fidelity, and table reconstruction?

RQ5 — Consistency. Do the performance differences between strategies hold consistently across different thesis documents, or does the relative ranking between strategies depend on the specific document?

These questions are tested against four hypotheses, which are evaluated against the experimental results in Chapter 6.

H1 — Adaptive superiority. On documents with diverse page content, the adaptive system should outperform any single fixed strategy applied uniformly, because it matches the best-suited strategy to each page type. On documents with only one content type, adaptive should equal the corresponding fixed strategy rather than exceed it.

H2 — Strategy-content fit. Text-only will work well on plain-text pages but fail on image-heavy or formula-rich content. Image-only and hybrid strategies should handle diverse content well. Both treat every page as an image and pass it through a vision LLM regardless of what the page actually contains, which produces higher coverage but at significant computational cost.

H3 — Efficiency of the adaptive approach. Despite the extra classification step, the adaptive system should cost less on average than image-only or hybrid. Plain-text pages skip the LLM entirely, and non-text pages receive targeted prompts rather than a single general-purpose one, which reduces unnecessary inference.

H4 — Hybrid as a strong baseline. For documents with a balanced mix of text and visual content, the hybrid strategy should be a competitive baseline. The adaptive system is expected to match or beat it by avoiding unnecessary LLM calls on plain-text pages.

1.4 Scope and Delimitations

This thesis works only with PDF as the input format. The system is tested on bachelor thesis documents in German and English. The system architecture is compatible with any LLM backend that supports a vision Application Programming Interface (API). All experiments use LM Studio as a self-hosted runtime rather than a third-party cloud AI service, which keeps results reproducible regardless of which machine LM Studio runs on; Section 4.4 describes the hardware setup used for the experiments.

The system detects the document language from the first page using the `langdetect` library and adjusts LLM prompts to prevent the model from translating content. If the page text is too short for reliable detection, the system defaults to English. No language-specific training or fine-tuning is applied, and the evaluation metrics are language-agnostic.

The following are out of scope: real-time or streaming document processing, input formats other than PDF (e.g., DOCX, HTML, PPTX), fine-tuning or task-specific training of LLM models, enterprise-scale deployment, cloud infrastructure, and support for right-to-left scripts or ideographic writing systems. The output format is Markdown only. Conversion to other structured formats such as LaTeX, HTML, or JSON is not evaluated here, though the architecture could support it through prompt changes.

1.5 Thesis Contributions

The central contribution is a per-page content classifier that assigns each PDF page to one of six types: TEXT, IMAGE, FORMULA, TABLE, MIXED, or EMPTY. It uses structural signals from PyMuPDF: embedded image count, vector path density, presence of mathematical Unicode symbols, table detection, and character count. No LLM is involved at this stage, so classification is fast and deterministic.

For text pages, heading levels are inferred from font sizes relative to the dominant body text size on the page. For pages routed to the vision path, embedded figures are extracted using PyMuPDF’s image extraction API and saved as individual files. The vision LLM is instructed to include these paths as Markdown image links, so figures are preserved in the output.

Output quality is measured across seven metrics: text similarity, heading structure, list structure, table structure, code block score, paragraph structure, and word overlap. These cover aspects of structural fidelity that BLEU and ROUGE cannot capture. A configurable experiment runner tests all combinations of strategy, model, prompt variant, temperature, and page across the benchmark documents, records scores and token usage per page, and generates comparison reports.

The full system runs locally using PyMuPDF, pymupdf4llm, and LM Studio. All prompt templates, evaluation scripts, experiment configurations, and test suites are included in the repository.

1.6 Thesis Structure

The thesis builds in three phases. The first establishes the foundations and situates the work. Chapter 2 covers the technical background: how PDF stores content internally, what Markdown is and why it suits this use case, how modern LLMs process documents, and how the three main extraction strategies differ. Chapter 3 surveys existing tools and systems, from traditional PDF parsers to multimodal AI models, and identifies the gap this work addresses.

The second phase covers the research design and implementation. Chapter 4 describes the methodology: the document dataset, ground truth construction, evaluation metrics, and experimental setup. Chapter 5 documents the system architecture, from the per-page classifier and four conversion strategies to the LM Studio integration and post-processing pipeline.

The third phase presents the findings and their interpretation. Chapter 6 reports the experimental results, with scores by strategy, model, and document, and qualitative analysis of where each strategy succeeds or fails. Chapter 7 interprets those results against the five research questions and four hypotheses, and examines threats to validity. Chapter 8 closes with a summary of contributions, limitations, and directions for future work.

Chapter 2

Background and Fundamentals

Choosing the right extraction strategy for a PDF page requires understanding both how PDFs encode content and what different extraction methods are capable of. This chapter provides that context.

2.1 PDF Format and Parsing Challenges

PDF evolved from PostScript, a page description language built for rendering, not for storing structured information [1]. Because of this origin, PDF encodes content as drawing instructions rather than semantic elements. Text is stored as individual glyphs placed at (x, y) coordinates on the page, with no native concept of paragraph, heading, or list. The page origin sits at the bottom-left corner with Y increasing upward, which is the opposite of screen coordinates. This means any tool trying to recover reading order must first apply a coordinate transformation, and even then the result fails on multi-column layouts [1]. Any structure a reader perceives has to be rebuilt from geometric cues alone, which is unreliable in practice [14].

The ISO 32000-1 standard introduced “tagged PDF” in version 1.4. Tagged PDFs include a logical structure tree that maps rendering commands to semantic roles such as headings and paragraphs and specifies reading order [9]. But most PDFs in practice are untagged, so structure must be guessed from geometry alone. And even when tags are present, correct reading order still requires manual verification [9]. There is also an important distinction between *native PDFs*, which contain actual text objects with bounding boxes that can be parsed directly, and *scanned PDFs*, where pages exist only as images and need OCR and visual region detection before any text can be extracted [1]. Deciding which category a page belongs to is itself a preprocessing step, usually done by comparing the ratio of embedded text area to image area. To make things more complicated, some digitally produced PDFs contain visually correct but internally garbled text encodings, which means they need an OCR fallback even though they are not scanned [1].

Text extraction from native PDFs has its own set of problems. PDF can embed fonts with custom encoding vectors, so character codes in the file do not always map to Unicode. This is a common cause of garbled or incomplete extraction, especially in older documents [1]. Even when characters are recovered correctly, simple sequential extraction still loses paragraph boundaries, list structure, and semantic grouping [2]. Multi-column layouts, footnotes, and mixed-language documents make this worse, because positional order no longer corresponds to reading order [14]. Formulas are a particular problem: PDF stores them as sequences of individual characters at specific coordinates, so superscripts, subscripts, and multi-line expressions have no semantic markup and are often merged with body text by extractors [8]. Figure 2.1 shows two of these effects on a real page from the corpus used in Chapter 4: the chapter heading arrives as two plain text lines with nothing marking them as a heading, and words are cut in half wherever the print layout hyphenated them at a line end.

Raw sequential extraction (PyMuPDF)

```
1 Chapter 1
2 Introduction
3 Systems that automate the scientific research process, including searching
  literature, gen-
4 erating hypotheses, running experiments and writing manuscripts, are no longer
  just con-
5 cepts. Systems like data-to-paper [1], The AI Scientist [2] or Agent Laboratory
  [3] have
6 ...
```

The structure the reader perceives (ground-truth Markdown)

```
1 # Chapter 1: Introduction
2 Systems that automate the scientific research process, including searching
  literature, generating hypotheses, running experiments and writing
  manuscripts, are no longer just concepts. ...
```

Figure 2.1: Sequential text extraction versus intended structure, shown on the first body page of one evaluation document. The raw extraction keeps the visual line breaks, including mid-word hyphenation, and carries no heading markup.

Images embedded in a PDF carry no context linking them to the surrounding text, so their relationship to a caption or paragraph is lost during extraction [14]. Tables are similarly problematic: there is no explicit row or column structure in PDF, and complex merged cells often require HTML (HyperText Markup Language) output rather than Markdown, since Markdown has no native support for `rowspan` and `colspan` [6]. Code blocks can be identified through monospaced font detection and consistent horizontal spacing, but

standard text extractors tend to fragment them because they do not preserve whitespace-sensitive formatting [2].

2.2 Markdown Format

Markdown’s syntax uses standard ASCII characters: # symbols for headings, hyphens or asterisks for lists, backticks for inline code, and triple backticks for fenced code blocks. Because the syntax is so simple, a Markdown file is readable without any renderer, which is not the case for binary or XML-based formats like PDF or DOCX.

Several elements in this thesis use GitHub Flavored Markdown (GFM) extensions, including pipe-delimited tables and inline $\text{\LaTeX}$ math. Table 2.1 lists the most commonly used Markdown syntax elements referenced in this thesis.

Table 2.1: Common Markdown syntax elements

Element	Syntax	Description
Heading 1	# Heading	Top-level section heading
Heading 2	## Heading	Second-level heading
Heading 3	### Heading	Third-level heading
Bold	**text**	Bold emphasis
Italic	<i>*text*</i>	Italic emphasis
Unordered list	- item	Bullet list item
Ordered list	1. item	Numbered list item
Inline code	`code`	Monospace inline snippet
Code block	``lang ... ``	Fenced block with optional language tag
Link	[text](url)	Hyperlink
Image	![alt](path)	Embedded image reference
Table	col col	GFM pipe-delimited table
Math (inline)	$\$formula\$$	Inline $\text{\LaTeX}$ expression (GFM extension)

Markdown preserves semantic hierarchy through headings, nested lists, emphasis, and tables, which makes the output usable by both human readers and automated tools. The GitHub Flavored Markdown (GFM) extension adds support for tables, task lists, and $\text{\LaTeX}$ -style math notation [11], which covers most of what academic and technical documents

require. This means the output of a PDF-to-Markdown conversion can be fed directly into documentation platforms, static site generators, or AI pipelines without any extra transformation step.

2.3 Large Language Models

Large language models (LLMs) are neural networks trained on large text corpora to predict and generate text. Almost all modern LLMs are built on the transformer architecture, which uses self-attention as its core mechanism. Self-attention lets each token in a sequence attend to every other token, regardless of how far apart they are. This is what allows LLMs to handle long documents and maintain coherence across a full text. Training on diverse data (books, articles, code, and web text) gives these models broad knowledge and the ability to perform many language tasks: translation, summarization, question answering, and code generation, among others.

One practical constraint is the *context window*: the maximum number of tokens a model can process in a single inference call. Larger context windows allow the model to consider longer documents without losing earlier content, but they also require more memory and compute [5]. For document conversion, the context window determines how much of a page, or how many pages at once, can be submitted in a single prompt.

LLMs vary in capability and cost. Proprietary models like GPT-4.1 combine text and visual inputs in a single end-to-end network. Open-source alternatives like Meta’s LLaMA 3.3 and Mistral Large reach competitive performance at much lower inference cost, which makes them practical for local deployment via runtimes like LM Studio.

Three parameters control how an LLM generates text and are configurable in local runtimes such as LM Studio [12]. *Temperature* sets how much randomness is applied when the model picks the next token. At temperature 0, the model always picks the most probable token, so the output is the same every time for the same input. Higher values make the model sample more broadly — useful for creative tasks, but a problem for document conversion where consistency matters. *Top-K* limits candidates to the K most likely tokens at each step. *Top-P* (nucleus sampling) works differently: it selects from the smallest group of tokens whose combined probability reaches a threshold P [12]. At $P = 0.9$, only the tokens that together cover 90% of the probability mass are candidates, and the size of that group shifts at each step based on how confident the model is. For document conversion, temperature 0 is the right setting. It removes randomness, ensures the same page produces the same output on every run, and makes evaluation results repeatable.

A specific class of models has been developed for visually rich document understanding (VRD-CU). These models treat documents as multimodal objects where text, spatial layout, and visual features all carry meaning together. The LayoutLM family [13] is a well-known example: these models are pretrained on text content, two-dimensional positional embeddings that encode word locations on the page, and visual features from page images. This joint training lets the model recognize a heading not just from its words, but from its position, font size, and surrounding whitespace — signals that a text-only model cannot see. The field has gone through three generations: early feature-based models (2018–2020), BERT-style document models (2020–2022), and LLM-instruction-tuned frameworks (2022–present) [13], which shows how quickly multimodal document understanding has matured as a research area.

2.4 Multimodal Processing Strategies

PDF parsing approaches fall into three broad categories based on how they interpret document content: rule-based text extraction, deep learning-based recognition, and multimodal processing [5]. A complete parsing system has to handle five interdependent tasks: layout analysis, text recognition, table structure recognition, mathematical expression recognition, and chart understanding [7]. No single approach handles all five, and the difficulty of each task varies depending on the document type. This is one of the main reasons fixed-strategy tools struggle on diverse real-world documents.

Rule-based extraction reads the PDF’s embedded text layer directly, retrieving character sequences and their coordinates without any visual processing. Libraries like PyMuPDF and pdfplumber work this way. They are fast, deterministic, and reliable on native PDFs with simple layouts, but they cannot process scanned or image-based pages at all and they discard all visual structure [2].

Deep learning approaches use trained neural network models to recognize text, layout regions, and graphical elements from page images. Tesseract is a well-known example; more recent tools use deeper architectures for higher accuracy across a wider range of content types. These approaches can handle scanned documents that rule-based tools cannot, and they outperform multimodal tools on pages dominated by complex images [5]. But applying them to native PDFs that already have an extractable text layer introduces unnecessary noise and discards existing formatting metadata [2]. Multi-stage pipelines also have an error propagation problem: a wrong classification in the layout analysis step affects every downstream stage, and the resulting errors are hard to trace back to their source [11].

Multimodal extraction combines the text layer with visual understanding, using models that process both raw text and a rendered image of the page at the same time. By taking rendered page images as input, these models can detect visual structure that text-layer extraction misses entirely: reading order, column boundaries, figure captions, and mathematical notation. Prompt instructions can guide this further — specifying the output format, document language, and page content type pushes the model toward structured, consistent output. These systems can extract text, images, and tables in a single pass and align them according to the document’s logical structure [5]. Ingemarsson and Persson [5] found that multimodal tools achieved the best overall performance across diverse document types, reaching 90% accuracy with GPT-4o at the time of publication. This makes them the strongest choice when the page content is unknown or mixed.

All three strategies involve trade-offs. Rule-based methods are fast and reliable for text but cannot handle images. Deep learning methods handle visual content better but may over-process native PDFs. Multimodal methods offer the broadest coverage but at higher computational cost. No single strategy is the right choice for every page. Recent benchmarks support this: Ouyang et al. [4] benchmarked pipeline-based and end-to-end multimodal systems across diverse document layouts in OmniDocBench and found that no single system performs consistently well across all content types. Structural failures persisted across all tested parsers, particularly for charts and complex visual layouts.

Chapter 3

Related Work

The gap this thesis addresses does not exist in isolation. It sits at the end of a line of work on PDF processing tools, modern conversion systems, and LLM-based document understanding. Each of these areas has pushed the field forward, and each has a limitation that the next tried to fix. This chapter traces that line and identifies where it stops.

3.1 Traditional PDF Processing Tools

Traditional PDF processing tools fall into two categories: text-layer parsers that read content directly from the embedded glyph stream of a native PDF, and OCR-based tools that recover text by applying image recognition to rendered page images. Both are well-established, but they were built to extract text, not to reconstruct the semantic structure of a document. That is the core limitation both categories share.

3.1.1 Text-based Parsers

Text-layer parsers like PyMuPDF, pdfplumber, and pypdfium2 read character objects and their coordinates from a PDF's internal object stream, then reassemble them into reading order using heuristics. Each library works at a different level of detail. PyMuPDF gives direct access to character bounding boxes, font metadata, and color attributes, which makes it fast and useful as a preprocessing step. pdfplumber is built on PDFMiner and exposes bounding-box tables for words and characters, which helps with positional grouping for tabular data. pypdfium2 wraps the PDFium rendering engine (the same one used in Google Chrome) and produces high-fidelity rendering, but does little structural post-processing on its own.

Meuschke et al. [2] benchmarked ten tools on 500,000 pages of academic documents and measured F1 scores across element types. The results are specific. PyMuPDF scored $F1=0.51$ on paragraph extraction because it assigns sections, captions, lists, and equations

all to the same paragraph label, producing noisy output. Table extraction was poor across every tool: the best result was $F1=0.47$ (Adobe Extract), while dedicated table parsers like Camelot and Tabula reached only 0.28–0.30, partly because they misidentify two-column layouts as tables. List extraction scored zero for every tool tested. Equation extraction was near zero. These are not edge cases — they are common elements in academic and technical documents. Ingemarsson and Persson [5] identified a further hard limit: on pages where content exists only as rasterised images, text-layer parsers recover nothing. For mathematical expressions, Gao et al. [14] show that sequential character extraction loses the spatial relationships that make formulas meaningful. The output is character-correct, but the structure is gone.

3.1.2 OCR Solutions

OCR tools work differently: instead of reading a text layer, they apply image recognition to rendered page images. Smith [3] describes the architecture of Tesseract, the most widely used open-source OCR engine. It runs a two-pass pipeline: the first pass uses adaptive thresholding and connected-component analysis to isolate character candidates, and the second pass applies a word-level language model to resolve ambiguous boundaries. Tesseract 4.0 added a Long Short-Term Memory (LSTM)-based recognition model that improved accuracy on degraded and stylised text while keeping the same two-stage structure. Subramani et al. [7] reviewed deep learning approaches to document understanding and placed classical engines like Tesseract in that broader context. Their conclusion is that single-pass OCR works well on clean, single-font documents but breaks down on complex layouts, mixed-script content, and low-resolution scans. This is what pushed the field toward layout-aware, multi-stage pipelines.

Zhu et al. [6] developed DocBed, a multi-stage OCR pipeline built for documents with complex layouts. Rather than applying recognition to the full page at once, DocBed first runs a region segmentation model to separate text blocks, tables, and figures, then sends each region to a dedicated recogniser. This staged approach recovered significantly more structure than global OCR on layout-complex documents. But it also introduced a known failure mode: if the segmentation step makes a wrong decision, it passes incorrect bounding boxes to every recogniser that follows, and the resulting errors are hard to trace back to their source. This is not a problem unique to DocBed. It applies to any modular pipeline architecture [11].

3.1.3 Shared Limitations

Both text-layer parsers and OCR tools share a gap that Darvishy [9] identified: no traditional tool can interpret or describe mathematical formulas or scientific graphs in PDFs. Ouyang et al. [4] show this extends even to specialised formula tools: exact-match rates are near zero despite high character-level scores, because structural grouping is lost even when individual symbols are correct. Figures fare no better: every tool in their benchmark omitted figure content from structured output entirely. Multi-column layouts are a third shared weakness: reading-order error roughly doubles on complex layouts compared to single-column pages across every tool category tested [4]. For the academic and scientific documents this thesis targets, formulas, figures, and multi-column layouts are not edge cases. The evidence from Meuschke et al. [2], Ingemarsson and Persson [5], and Subramani et al. [7] all point to the same conclusion: traditional tools work well within a narrow range of document types, but they are not sufficient for the diverse, structure-rich documents this thesis works with. This is what motivated the move toward learning-based and multimodal approaches.

3.2 Modern Document Conversion Tools

Modern conversion tools go beyond simple text extraction. Rather than reading the raw text layer, they first analyse the visual structure of each page. This approach is sometimes called pipeline-based document parsing: the page is decomposed into layout blocks, each classified as text, table, figure, or heading, and the blocks are then processed in reading order [1]. Scanned pages are detected automatically and routed to an OCR module, while digitally-born pages go through a text extraction path. Subramani et al. [7] survey these methods, distinguishing between modular pipeline systems and unified Vision-Language Model (VLM) approaches. Robust handling of complex layouts remains an open challenge for both.

Commercial tools such as LlamaParse, Marker, and Mistral OCR follow this general architecture but add API layers and post-processing for downstream use cases. Ouyang et al. [4] evaluated these tools across diverse real-world document layouts and found no single system performs consistently well across all content types.

Blecher et al. [8] introduced Nougat, a Visual Transformer trained on paired PDF-markup data that converts full academic document pages directly to Markdown. Nougat removes the need for a separate layout analysis or OCR stage by generating structured output end to end. It handles headings, equations, tables, and citations from a single model pass.

Subramani et al. [7] position this shift in a broader context: it represents the move from pipeline-based recognition to unified visual models that reason over the full page at once.

For long, multimodal academic documents, Xie et al. [15] introduced PDF-WuKong, a multimodal large language model for question-answering over long PDFs. The system uses a sparse sampler that operates on both text and image representations, selecting only the paragraphs and diagrams most relevant to a given query. Evaluated on the PaperPDF dataset of 1.1 million QA pairs from English and Chinese academic papers, PDF-WuKong outperformed proprietary products by an average of 8.6% on F1. This shows that efficient joint modelling of text and images is achievable for long documents, though the system targets retrieval and question-answering rather than full document conversion.

Even so, key limitations remain. Meuschke et al. [2] found that even modern parsing tools struggle with scientific and patent documents containing dense formulas and non-standard layouts.

3.3 LLM-based Document Understanding

Before large language models, document understanding depended on chains of specialised tools. Vision-capable language models changed this: they read a rendered page image and produce structured output token by token through autoregressive generation, with no hand-coded layout rules in between. Nougat [8], described in the previous section, is the clearest example: Meuschke et al. [2] confirmed that structured-output tools of this kind outperformed all rule-based parsers on scientific documents.

OpenDataLab [16] released MinerU, an open-source dual-engine system that automatically detects whether each page is scanned or digitally born and routes it to a Vision-Language Model or a classical OCR path. It supports 109 languages and outputs structured Markdown or JSON (JavaScript Object Notation). Duan [11] pushes the argument further: end-to-end decoder transformers outperform pipeline-based systems on accuracy because they never pass an incorrect intermediate result down a fixed processing chain.

Persistent gaps remain across all approaches: word-level representations miss document-level semantic structure, and multi-page documents with mixed content are still an open challenge [13].

No current model covers all content types reliably. This gap motivates the image-based strategy in this thesis, where rendered page images are passed to a multimodal LLM and a postprocessing pipeline corrects structural failures in the output.

Table 3.1 compares the tools discussed in this chapter on the dimensions most relevant to structured document conversion. *Partial* in the Formulas column means formula pages are routed to a vision model but the LaTeX output is not validated; *Partial* in the Routing column means the tool routes between two fixed paths rather than by content type.

Table 3.1: Comparison of PDF processing tools across dimensions relevant to structured document conversion

Tool	Input	Formulas	Tables	Per-page routing	Output
PyMuPDF	native	No	Partial	No	text
Tesseract	scanned	No	No	No	text
Nougat	native	Yes	Yes	No	Markdown
MinerU	both	Yes	Yes	Partial	Markdown
This work	both	Partial	Yes	Yes	Markdown

The *both* label for this work reflects the image-path strategies: text-only pages go through PyMuPDF’s text layer, while all other page types are rendered as PNG images and passed to the vision-language model. A scanned PDF would follow the same image path and the VLM would read the rendered scan. However, the page classifier relies on text-layer signals (character count, table detection, formula detection), which are absent in scanned documents. The evaluation in this thesis covers native PDFs only, and the classifier thresholds were calibrated on native content.

3.4 Evaluation Methods for Document Conversion

Evaluating document conversion is harder than evaluating plain text extraction. The output combines headings, paragraphs, tables, formulas, and figures, and each element type can fail in a different way. Early work used string-similarity metrics: BLEU score and edit distance (Levenshtein similarity) both compare extracted text against a reference string, character by character [17]. Meuschke et al. [2] added local alignment as a third metric, which allows gaps between matched segments and catches word-order errors, such as merged columns or reversed reading order, that BLEU misses.

All three metrics treat the full output as a flat sequence. A table with one column shifted receives a low Levenshtein score even when nearly all the content is correct. Tree Edit Distance based Similarity (TEDS) addresses this for tables: it compares the predicted tree structure against a reference and gives partial credit when individual cells are right even if

the overall structure differs. Research in document parsing evaluation has shown that no single metric covers all content types, with layout analysis using mean Average Precision (mAP), text extraction using edit metrics, and table evaluation using TEDS [18, 19].

Aggregate text-similarity scores hide capability gaps. A parser may score well under BLEU or edit distance but have a misaligned table header or a dropped superscript that breaks any downstream process. Ouyang et al. [4] benchmarked diverse parsers across multiple content dimensions and found that no system performed well on all types, confirming that a single aggregate score cannot capture document parsing quality.

For conversion to Markdown specifically, N-gram metrics carry an additional limitation: they ignore syntax. A heading marked with # and the same text written as bold have near-identical character content but different semantic meaning in the rendered document. This is why the evaluation in Chapter 4 scores structural, content, and formatting dimensions separately rather than collapsing them into one number.

Chapter 4

Methodology

Comparing four extraction strategies fairly requires a controlled setup: the same documents, the same scoring framework, and a baseline simple enough to serve as a meaningful lower bound. Figure 4.1 shows the complete evaluation pipeline.

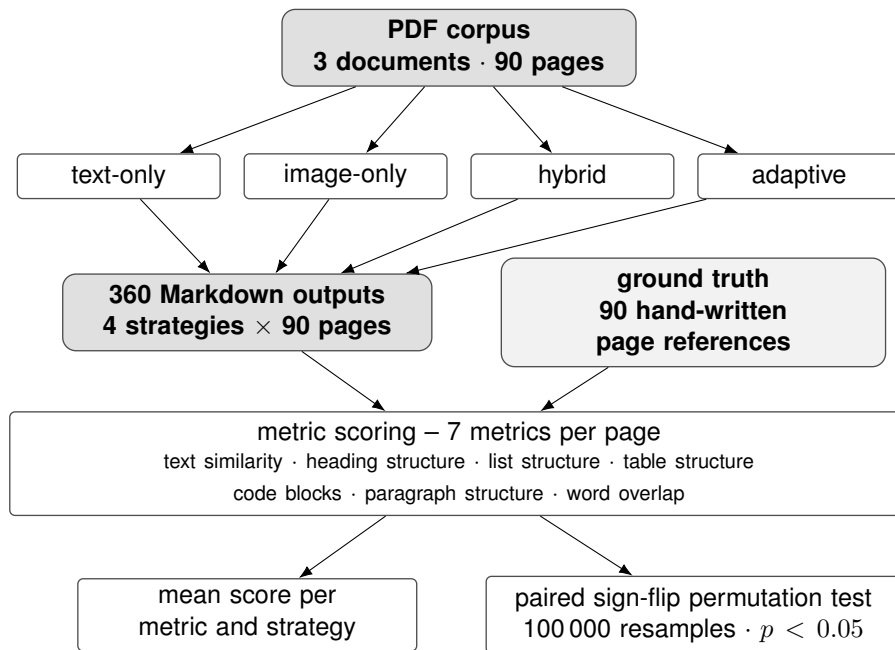


Figure 4.1: Evaluation pipeline: corpus, strategies, ground truth, metrics, mean scores, and the paired permutation test

4.1 Research Design

This thesis uses a comparative study to evaluate four PDF extraction strategies against a shared test corpus and scoring framework. The comparative approach is standard for PDF parsing research. Meuschke et al. [2] applied it to benchmark PDF tools across multiple document domains. Ingemarsson and Persson [5] used the same design to rank model

families by accuracy and computational cost. Using a shared methodology means the results sit alongside that published work rather than in isolation.

The four strategies are text-only, image-only, hybrid, and adaptive. The first three are fixed: every page goes through the same pathway regardless of content type. The adaptive system classifies each page first, then routes it to the appropriate extraction method. This is the only design that can skip the LLM entirely on plain-text pages. That is what RQ3 tests.

The five research questions each target a different aspect of this comparison. RQ1 and RQ2 compare accuracy across all four strategies: RQ1 asks whether the adaptive system beats every fixed strategy, and RQ2 asks how the fixed strategies rank against each other across document categories. RQ3 covers efficiency: processing time, token usage, and LLM call count per strategy. RQ4 drives the metric design in Section 4.3. RQ5 checks whether the strategy rankings stay consistent across different thesis documents.

Markdown is the target output format for all strategies. Unlike PDF or Word, it stores structure in plain text, requires no proprietary software to read, and works directly with LLM and RAG systems [10, 12]. Table 4.1 compares all three formats across eight properties.

Table 4.1: Comparison of PDF, Word (.docx), and Markdown as document formats

Property	PDF	Word (.docx)	Markdown
Human readable (raw)	No	No	Yes
Machine parseable	Difficult	Moderate	Easy
Version control (Git)	Poor	Poor	Excellent
Format portability	High	Low	High
Preserves visual fidelity	Excellent	Good	Limited
LLM/RAG compatibility	Poor	Moderate	Excellent
Requires proprietary software	Partial	Yes	No
Long-term archival stability	High	Moderate	High

4.2 Dataset and Ground Truth Preparation

The test corpus consists of bachelor thesis documents. This scope was chosen because academic theses are structurally demanding: they combine dense body text, mathematical formulas, multi-column layouts, citations, tables, and figures, often within the same

document. They also represent the primary use case this system targets. Testing across multiple thesis documents rather than a single one allows RQ5 to check whether strategy rankings are stable across documents or vary by the specific content mix of each thesis.

Documents were drawn from real sources rather than synthetic benchmarks. Meuschke et al. [2] found that tool performance varies significantly by document structure, which is why the corpus was built to include theses with different content mixes and layouts. The documents are primarily in English, with one document in German, reflecting the intended usage context. Each selected document contains at least one structurally challenging page: a formula-heavy section, a multi-column spread, or a table with complex cell structure. The final corpus consists of three thesis documents totalling 90 evaluated pages, 30 per document. All three belong to a single document category: academic bachelor theses. This single-category design keeps the comparison focused on the intended use case but limits the scope of the results to this document type. Three documents is a small corpus. The results support strategy comparisons within this set, but are not sufficient for general claims about other document types, multilingual content, or different academic formats. This limitation is discussed further in Chapter 8.

Each test document has a corresponding reference Markdown file, created by hand. Each reference captures the full document structure: heading hierarchy, paragraph flow, table cells, list nesting, and code blocks where present. Pfitzmann et al. [19] argue that human-created annotation is the only reliable baseline for evaluating document parsing tools, because automated references embed the same systematic errors as the tools being tested. Creating the reference by hand ensures that the evaluation measures real output quality rather than agreement between two imperfect automated systems.

4.3 Evaluation Metrics

RQ4 asks how Markdown quality can be measured beyond BLEU and ROUGE. Section 3.4 showed that flat sequence metrics reward correct content in the wrong structure. The answer is to score each element type separately. Seven metrics are used, grouped into three categories: content accuracy, structural accuracy, and formatting accuracy.

Two metrics measure how much text was preserved. Text similarity uses difflib’s `SequenceMatcher`, which compares candidate and reference character by character and returns a ratio between 0 and 1. Word overlap computes the Jaccard similarity on the word sets of both outputs. Text similarity catches character-level noise such as OCR artefacts. Word overlap catches word-level omissions such as missing paragraphs.

Three metrics target document structure. Heading structure score compares the candidate and reference heading counts at each level from H1 to H6, skipping levels that are absent in both. List structure score compares the number of list items, bulleted and numbered combined, between candidate and reference. Paragraph count score compares paragraph counts with a 20 per cent tolerance: a candidate within that range of the reference scores 1.0, with a linear penalty beyond it. A document can have high text similarity and still fail all three of these if the extractor collapses the heading hierarchy into plain text.

Two metrics cover formatting elements. Table detection score checks for the presence and count of pipe tables. Code block score checks for fenced code blocks. Both are low-frequency elements, but a single missed table is a meaningful failure for any document that contains one. These metrics make that failure visible rather than diluted by high text similarity on the surrounding prose.

TEDS (Tree Edit Distance based Similarity) offers finer-grained table evaluation by comparing cell-level tree structure [18]. It was considered but not used. The test corpus does not include tables complex enough to require cell-level scoring, and producing cell-level annotations by hand for a multi-document corpus carries a high annotation cost. Table detection score covers the relevant case for this evaluation.

Score differences between strategies are tested for statistical significance with a paired sign-flip permutation test. Every page is converted by all four strategies, so per-page scores form matched pairs, and the test asks how often a mean difference at least as large as the observed one would arise if each page’s difference were equally likely to carry either sign. The test is two-sided, uses 100,000 resamples, and treats $p < 0.05$ as significant. It makes no assumption about how the scores are distributed, which matters for metrics where most pages tie and only a handful differ.

4.4 Experimental Design

The experiment applies each of the four strategies to every document in the corpus at a fixed temperature of 0.0. Deterministic output is required here because the evaluation compares strategies on the same pages: any randomness introduced by higher temperatures would obscure whether differences in score come from the strategy or from sampling noise.

One vision-language model is used across all VLM strategies: Qwen2.5-VL 7B, served through LM Studio on a rented cloud GPU rather than local hardware. The model runs locally just as well, but a full 360-page run on local hardware takes hours; the same run completes in minutes on the rented GPU, which is why it was used here. The text strat-

egy does not use a model at all. It calls PyMuPDF directly through the `pymupdf4llm` library, which converts the PDF text layer to Markdown without invoking any LLM. The three vision-based strategies therefore run under identical hardware and model conditions, which keeps that part of the comparison fair. Text-only’s processing time instead reflects local disk and CPU speed, since it never reaches the GPU at all. Each strategy’s output is scored after the full postprocessing pipeline described in Section 5.5, so the metrics in Chapter 6 measure what the deployed system actually returns to a user, not an earlier, unpolished stage of it.

Text-only extraction at temperature 0.0 is the baseline. It is the simplest possible extraction method: raw text from PyMuPDF with no image input and no LLM call. A strategy must outperform it on at least two metric categories to justify its additional computational cost. Processing time is recorded per page, providing the raw measurements for RQ3. Table 4.2 summarises the expected strengths and weaknesses of each approach based on the literature [1, 2].

Results are reported as mean scores per document across all pages in that document. Meuschke et al. [2] used the same document-level reporting and found that tools which rank well on one document often rank poorly on another. Document-level means make this pattern visible. A strategy that wins overall but fails on specific documents is a weaker result than one that performs consistently across the full corpus.

Table 4.2: Preliminary comparison of PDF extraction strategies

Criterion	Text-only	Image-only	Hybrid	Adaptive
Works on scanned PDFs	No	Yes	Yes	Yes
Preserves reading order	Partial	Yes (model-dep.)	Yes	Yes
Handles complex tables	Poor	Moderate	Good	Good
Handles mathematical formulas	Poor	Good	Good	Good
Computational cost	Low	High	Medium-High	Low-Medium
Structural fidelity	Low	Moderate	High	High
Dependency on LLM	None	Full	Partial	Selective

Chapter 5

System Design and Implementation

The system was designed around one principle: avoid invoking the LLM when the text layer alone is sufficient. This chapter documents how that principle shapes each component, from the page classifier that decides when the LLM is needed to the postprocessing pipeline that cleans up what it produces.

5.1 Architecture Overview

The system takes a PDF file as input and produces one combined Markdown document as output. It consists of four modules: a page content classifier, a PDF processing module, an LLM integration layer, and a post-processing pipeline. Each module is implemented as a separate Python package and communicates through well-defined data structures. This keeps the components interchangeable and makes them testable without running the full pipeline.

Processing begins when a PDF is loaded by PyMuPDF [20]. For each page, the classifier analyses structural signals and assigns one of six content types: TEXT, IMAGE, FORMULA, TABLE, MIXED, or EMPTY. This classification determines which extraction path the page takes. The decision is made per page, so a single document can route different pages to different strategies.

TEXT pages are handled entirely by `pymupdf4llm` [21], which extracts the text layer and converts it to Markdown without invoking the LLM. All other page types are rendered as a base64-encoded PNG (Portable Network Graphics) image and passed to a vision-capable LLM. The LLM receives a structured prompt suited to the detected content type and returns a Markdown string. EMPTY pages are skipped, returning a short placeholder without any processing.

Pages are converted in parallel using a thread pool. The default concurrency is four workers, which overlaps the I/O wait time of LLM calls without overloading the LM Studio

server. Each worker opens its own `fitz.Document` handle, because PyMuPDF document objects are not thread-safe. The results are collected in page order and joined into a single Markdown document, with a horizontal rule between each page.

The LLM integration layer communicates with LM Studio [22] over a local HyperText Transfer Protocol (HTTP) server that exposes an OpenAI-compatible API [12]. All LLM calls are routed through a single `call_llm` function, which accepts the server URL, model name, message list, temperature, and token limit as parameters. Every strategy accepts this function as an injectable parameter, so tests can replace it with a fixed-response stub without touching the rest of the code.

The post-processing pipeline receives the joined Markdown and applies a fixed sequence of cleanup passes before writing the final output file.

Figure 5.1 shows the complete processing pipeline.

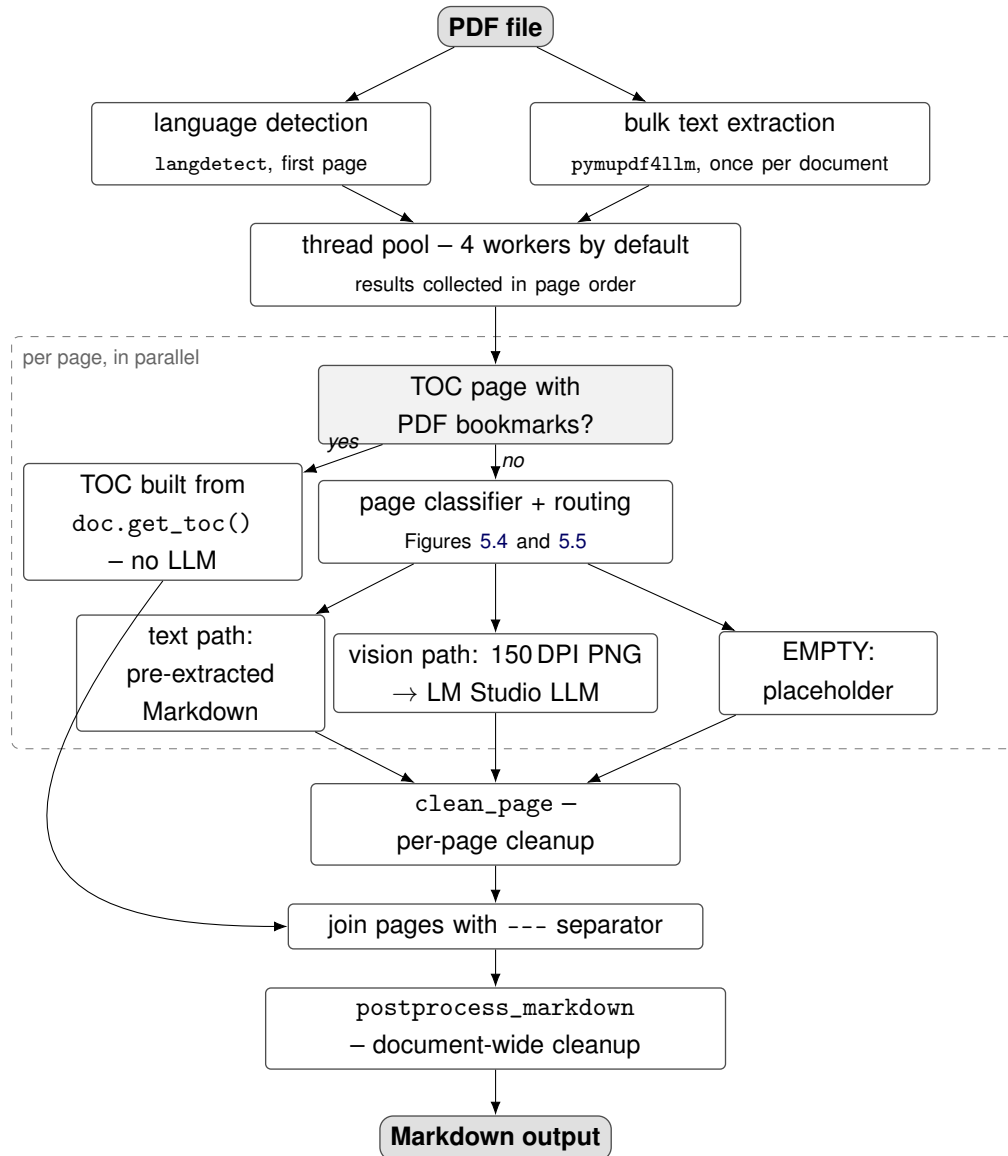


Figure 5.1: System architecture: from PDF input to Markdown output. TOC pages are rebuilt from the PDF bookmark outline and skip both the LLM and the per-page cleanup.

5.2 Page Content Classifier

The classifier extracts four structural signals from each page using PyMuPDF [20]: the number of embedded images, the number of vector drawing paths, the total character count of the text layer, and the number of detected tables. No machine learning is used. The classification is entirely rule-based, which keeps it fast and predictable.

Formula detection uses two independent checks. First, the classifier counts vector paths whose bounding box is smaller than 20 pixels in either dimension. More than 30 such

short paths on a single page indicates the presence of rendered mathematical symbols, since PDF renders formula components as small vector segments. Second, the classifier scans the text layer for mathematical Unicode characters such as $\sum$, $\int$, and ∇ , as well as LaTeX-style sequences that OCR engines sometimes preserve. Either check alone is sufficient to flag the page as formula-containing.

The classification follows a fixed priority order, working from the narrowest conditions toward the broader ones. The first check is for empty pages: fewer than ten characters, no images, and fewer than five vector paths. The next checks target the most visually distinctive content. A formula page has formula signals but fewer than 50 characters, since formula-only layouts tend to carry little surrounding body text. An image page has more than three embedded images, and a table page has at least one table detected by PyMuPDF's table finder. Pages that carry image or formula signals but did not qualify for any of those specific types fall into MIXED. Plain-text pages, those with 50 or more characters and no visual signals, are classified as TEXT. Any page that passes every check without a clear match falls back to MIXED with a confidence score of 0.5.

Each classification carries a confidence score between 0.5 and 0.95. TEXT pages score the highest at 0.95. To reach that classification, a page must have at least 50 characters and no image, formula, or table signals. That is the hardest combination to satisfy by accident. EMPTY and IMAGE pages score 0.9. EMPTY requires fewer than ten characters, no images, and fewer than five vector paths, which is a very narrow condition. IMAGE requires more than three embedded images, which rarely appears on a text-heavy page. FORMULA and TABLE score 0.85 because both detectors can misfire. Vector paths above the threshold sometimes come from decorative page elements rather than formulas. PyMuPDF's table finder occasionally flags dense grid layouts that are not real tables. MIXED pages score either 0.75 or 0.5. Pages that carry image or formula signals but did not qualify for IMAGE, FORMULA, or TABLE — for example, a page with one or two embedded images alongside substantial text — score 0.75. The fallback MIXED at 0.5 catches the remaining pages: fewer than 50 characters with no images, no formulas, and no detected tables. No signal was strong enough to assign a more specific type. The confidence score is stored in the PageAnalysis result but plays no role in the current routing logic. Pages are routed by type label alone, regardless of score. This is intentional: the rule-based classifier is deterministic, so the same signals always produce the same type and a score threshold would not change any routing decision. A future version could use the score to fall back to the hybrid strategy on genuinely ambiguous pages, where the MIXED 0.5 fallback fires.

Figure 5.4 shows the classification cascade. Figure 5.5 shows the routing checks that follow it.

5.3 PDF Processing Module

The PDF processing module has two sub-pipelines: one that extracts the text layer and one that renders pages as images. Which pipeline runs depends on the page type assigned by the classifier. TEXT pages go through text extraction only. IMAGE, FORMULA, and MIXED pages go through image rendering. EMPTY pages skip both. The hybrid strategy uses both pipelines on the same page.

TABLE pages are a special case: the system checks whether pymupdf4llm’s own text-layer extraction already produced a valid Markdown pipe table (detected by scanning for a `| -- | -- |` separator line) before deciding which pipeline to use. If it did, the page is treated like a TEXT page: no image is rendered and no LLM call is made, since pymupdf4llm’s table detector reads the native text layer directly and never paraphrases a cell value or drops the heading above the table. Only when pymupdf4llm’s extraction did not yield a usable table — typically a scanned page or a table embedded as an image — does the page fall back to image rendering with the vision model. This keeps the system’s total LLM cost down without sacrificing the pages that genuinely need the vision path.

Text extraction is handled by pymupdf4llm [21]. The system calls its `to_markdown()` function once for the whole document, with `page_chunks=True`, which returns one result per page as a list. This matters for performance: pymupdf4llm runs a full document scan to identify header font sizes on every call, regardless of how many pages are requested. Calling it once per page would repeat that scan N times, making extraction quadratic in the number of pages. Calling it once and indexing into the result list keeps extraction linear. Any inline figures found on TEXT pages are written to a configurable output directory, and their paths are corrected to be relative to the output file.

For the hybrid strategy, a second text extraction runs in parallel using a custom font-size-based extractor built on PyMuPDF [20]. This extractor builds a character-weighted tally of all font sizes on the page to identify the body text size, defined as the most frequent size by character count. Each line’s dominant font size is then compared to the body size as a ratio: lines at 1.8 times the body size or above are promoted to H1, 1.4 times to H2, and 1.15 times to H3. This produces a lightweight Markdown representation of the text layer that the LLM receives alongside the rendered image.

Image rendering converts each page to a base64-encoded PNG using PyMuPDF’s `pixmap` renderer. The render resolution is 150 dots per inch (DPI), set via a `fitz.Matrix` scale factor of $150/72$ in both dimensions, which is about twice the PDF native resolution of 72 DPI. The higher resolution preserves fine detail in tables, formulas, and diagrams, which improves LLM recognition accuracy. The encoded string is passed directly in the API request body as a data URI, so no temporary files are written to disk for vision pages. The render DPI is a configurable constant, so it can be raised for documents with very dense content or lowered to reduce memory use. Embedded figures can also be extracted separately using `extract_page_figures`, which saves each image to disk and returns relative Markdown paths that the LLM can reference in its output. The system also preserves the printed page label for each page using PyMuPDF’s `get_label()` method, which returns the label as it appears in the PDF (for example, “iii” for a Roman-numeral front-matter page rather than the 0-based index). These labels appear in the `<!-- Page iii -->` comments that separate pages in the output file.

5.4 LLM Integration

The system uses five prompt variants, each written for a specific content type. The *default* is a general-purpose prompt used when no more specific type is detected. The *text* variant is stricter: it instructs the model to assign heading levels by numbering depth, mapping chapter headings to #, sections to ##, and subsections to ###. It exists for an optional mode in which the text path’s raw output is passed through the LLM for cleanup; in the default configuration, text pages never reach the model at all, so this variant stays unused. Visual pages use one of three specialised variants. The *formula* variant instructs the model to transcribe all mathematical content as $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ notation, preserving subscripts, superscripts, and operators exactly. The *table* variant focuses on reconstructing pipe tables, with instructions to preserve all cell values and approximate merged cells in flat Markdown. The *diagram* variant asks the model to extract all visible text from figures and charts and add a short description of what the diagram conveys.

The message structure sent to the API differs by strategy. Text pages go to the API as a plain string in a single user message, with no image content. Vision pages are sent as multipart messages: the image strategy includes one or more base64-encoded images as `image_url` content blocks, with no text from the PDF. The hybrid strategy combines both, with the extracted text as the first content block and the page image following it. All strategies pass the selected system prompt in a system-role message before the user turn, using the OpenAI-compatible chat completions format (see Section 2.3).

When embedded figures have been extracted from the page in advance, as described in Section 5.3, a short text block is added to the user message listing them as Markdown image links. This instructs the model to include those links at the appropriate positions in its output rather than describing the figures with placeholder text. For non-English documents, a language-preservation note is appended to the system prompt by the `with_language_hint` function. The note names the document language and instructs the model to keep all text in the original language. This prevents the model from silently translating German or French content into English.

The adaptive strategy routes each page type to a specific prompt variant. FORMULA pages use the *formula* prompt, TABLE pages the *table* prompt, IMAGE pages the *diagram* prompt, and MIXED pages the general *default* prompt. TEXT pages are handled differently: the system calls the text strategy with `llm_call=None`, which returns the raw `pymupdf4llm` output without invoking the LLM at all. This saves the full round-trip cost on every plain-text page, which is the primary source of efficiency gain for the adaptive strategy on text-heavy documents.

Two safeguards keep this routing from losing content. Both appear in Figure 5.5. The first handles TEXT pages that are not as plain as they look. When the pre-extracted Markdown contains an image link, the page holds an embedded raster image the classifier's image count missed, most often a formula rendered as a small picture. Such a page is rerouted to the vision model so the formula becomes LaTeX instead of an opaque image. The images `pymupdf4llm` already saved are passed along as figure references, so they are not silently dropped. Table-of-contents pages are exempt, since the postprocessing TOC converter must receive them on the text path.

The second safeguard checks the vision model's answer before trusting it. A response shorter than 30 per cent of the page's own extractable text almost always means the model was cut off partway, not that the page has little text. The system then discards the vision output and falls back to the `pymupdf4llm` text, which is known to be complete. This check runs after the reroute above and on every MIXED page. A truncated model response can cost one page's formatting, but it can no longer cost the page's content.

5.5 Post-processing Pipeline

The raw output from `pymupdf4llm` and the LLM both contain predictable artefacts that must be removed before the Markdown is usable. The pipeline applies a fixed sequence of passes in two stages. The first stage runs per page via `clean_page()`, before individual

pages are joined into a document. The second stage runs on the full joined document via `postprocess_markdown()`. Separating the two stages matters because some artefacts, such as duplicate running headers, only become visible once multiple pages are combined.

The first group removes structural noise introduced by the PDF format. Running headers are repeated chapter or section titles that PDF places at the top of every page. `pymupdf4llm` extracts these as plain-text lines; the pipeline detects them by pattern and removes them from the first line of each page chunk.

The second group removes artefacts specific to LLM output. Decorative horizontal rules that appear directly after a heading are stripped; these come from PDF underlines misread as Markdown separators. Prompt instruction text that some models copy verbatim into their output is removed. ““markdown fences that some models place around their entire response are stripped: the fence is valid syntax for a code block but not for the top-level document.

The third group targets heading quality. `pymupdf4llm` often collapses all headings to the same level (`##`), so the pipeline re-assigns levels based on section-number patterns: a line matching `1.1.1 Title` becomes `###`, a line matching `1.1 Title` becomes `##`, and a standalone chapter title becomes `#`. A separate pass merges cases where the section number and its title were emitted as two consecutive heading lines at the same level. A further pass handles bare section numbers with no title text: the pipeline looks up the full title in the raw PyMuPDF text for the same page and patches the heading, recovering titles that the layout extraction missed. Headings that are single words with no recognised structural keyword are demoted to plain text.

The fourth group handles special content: table-of-contents pages, list pages, figures, and encoding. In the adaptive strategy, TOC pages are handled before per-page extraction runs. The pipeline reads the PDF bookmark outline directly via `doc.get_toc()` and formats it as a nested Markdown list with display page labels. The first TOC page receives this list; continuation pages are left empty. This bypasses `pymupdf4llm` entirely for TOC pages, avoiding the noisy pipe-table output it produces for these pages. Front-matter list pages (List of Figures, List of Tables, and Listings) are classified as TEXT and processed by `pymupdf4llm`, then a dedicated pass converts the pipe-table structure into clean Markdown and removes dot leaders. Figure caption lines are normalised to a single style: bold label, plain caption text. `pymupdf4llm` emits captions as bold lines, and the vision model sometimes emits them fully italicised; both forms become the same bold-label style used for source-code captions. Captions that appear before their image are swapped, blank lines between an image and its caption are removed, and label-only captions with no descriptive text are dropped. Figure 5.2 shows the swap on a corpus page: the vision model placed

the caption above the figure link, and the pass moves it directly below the image. The link target is the file that `extract_page_figures` saved to disk (Section 5.3), so this is also what a preserved figure looks like in the final Markdown. Italic spans containing only Greek letters are converted to inline LaTeX (e.g. an italic φ becomes φ). OCR superscript artefacts of the form `_EMC_[2]` are replaced with Unicode superscript characters.

Raw vision-model output

```
1 **Figure 1.1:** What is this Thesis about?
2
3 ![Figure 1](figures/page_009_fig_001.png)
```

After caption reordering

```
1 ![Figure 1](figures/page_009_fig_001.png)
2 **Figure 1.1:** What is this Thesis about?
```

Figure 5.2: Caption reordering on a corpus page (sentiment analysis thesis, page 9, classified as MIXED). The vision model emits the caption above the figure link. The pass swaps the two lines and removes the blank line, so the caption sits directly below its image. The link path is where `extract_page_figures` saved the embedded figure.

Source-code listings needed a detection path of their own. L^AT_EX’s listings package sets code in a fixed-width font. `pymupdf4llm` does not read that as a signal to build a code block. It either leaves the listing as plain text, or, when a numbered line looks like a heading, breaks it into fragments wrapped in scattered backticks.

The pipeline finds monospace lines directly from PyMuPDF’s font data. A font counts as monospace by name, matching substrings such as Courier or Consolas, or by measurement: if every character on the page has the same width, the font is fixed-width no matter what it is called. This second check exists because some PDFs give their code font a generic internal name with no clue in it at all. A page’s lines are only trusted as code if at least one shows real code syntax, such as a line-number gutter or a closing parenthesis before a colon. This rule keeps out bibliography entries and citation URLs, which are sometimes set in the same font as a listing but are not code.

A single page can hold more than one listing with prose between them. The reconstruction scans the page line by line and only groups consecutive matching lines into one fence. Prose and unrelated listings in between stay untouched. This stops two listings from merging into each other or swallowing the caption between them. Line numbers are stripped and renumbered in order, since L^AT_EX only prints a number for a line’s first physical row. A wrapped continuation gets no number at all, which looks broken once reflowed into one

fence. Figure 5.3 shows the pass on a corpus page. `pymupdf4llm` merges pairs of code lines into list items and leaves the line numbers inline. The reconstruction returns the listing as a fenced block, one code line per line, and the caption below it stays outside the fence.

Raw `pymupdf4llm` output

```

1 - 1 def getAction(state): # gibt -1 für Schläger links oder +1 für rechts
    zurück 2 global epsilon, Q
2
3 - 3 if np.random.rand() <= epsilon: 4 return np.random.choice([-1, 1])
4
5 - 5 return (np.argmax(Q[int(state)]) * 2) - 1
6
7 **Quellcode 5.2:** Methode getAction [8, S. 283]
```

After code block reconstruction

```

1 ```
2 1 def getAction(state): # gibt -1 für Schläger links oder +1 für rechts zurück
3 2 global epsilon, Q
4 3 if np.random.rand() <= epsilon:
5 4 return np.random.choice([-1, 1])
6 5 return (np.argmax(Q[int(state)]) * 2) - 1
7 ```
8
9 **Quellcode 5.2:** Methode getAction [8, S. 283]
```

Figure 5.3: Source-code reconstruction on a corpus page (neural networks thesis, page 21). `pymupdf4llm` emits the listing as list items that merge two code lines each; the monospace-font pass rebuilds it as a fenced code block and leaves the caption line untouched.

The same monospace check is reused during figure extraction. $\text{\LaTeX}$'s `listings` package draws a shaded box behind a listing. That box is a vector-filled shape, not a raster image. Earlier, `extract_page_figures` treated any large vector-drawn region as a possible figure. This mistook the shaded box for a diagram and saved a screenshot of the code instead of leaving it as text. The function now checks whether a candidate region's text is mostly monospace before treating it as a figure, and skips it if so.

A second problem appears when a page holds more than one vector-drawn element, such as two separate code boxes. Taking the bounding box of every vector drawing on the page and merging them into one region spans all the prose in between. `extract_page_figures` now groups vector drawings by vertical distance and checks each group on its own. Unrelated regions on the same page no longer merge into one oversized crop.

A last check filters out `pymupdf4llm`'s own fallback for complex equations. When it cannot read a formula as text, it renders that formula as a small image instead, usually under 50 pixels tall for one line of math. Carrying that image over to the vision model would duplicate the formula, since the model already converts it to $\text{\LaTeX}$ correctly from the page image. The height check drops the image before that duplication happens.

The second stage handles artefacts that only become visible on the joined document. Duplicate running headers that cross page boundaries are removed a second time. Lone page-number paragraphs, whether Arabic digits or Roman numerals, are stripped wherever they appear. Title-page metadata headings are demoted, and chapter headings split across pages are merged. Two further demotion passes run here: headings whose entire content is italic, and headings that are really numbered source-code listing lines, both become plain or list text. OCR text blocks that `pymupdf4llm` extracts from embedded figures are removed, since the figure is already included as a Markdown image link and the OCR output adds only noise. LaTeX math delimiters in the `\( . . . \)` and `\[ . . . \]` styles are normalised to the `$. . . $` and `$$ . . . $$` forms used throughout the output.

List pages get one more pass at this stage. `pymupdf4llm` sometimes renders the same list of figures or tables as plain bullets on one page and as table rows on the next, since it lays out each page on its own. A page-crossing pass rewrites the bullet entries into the same row format, so the whole list reads the same way from start to end. A second pass then merges table rows that a wrapped entry split into several pieces, joining them back into one row per entry and reversing any hyphenation that broke mid-word at the wrap point.

Figure 5.6 shows all passes in both stages.

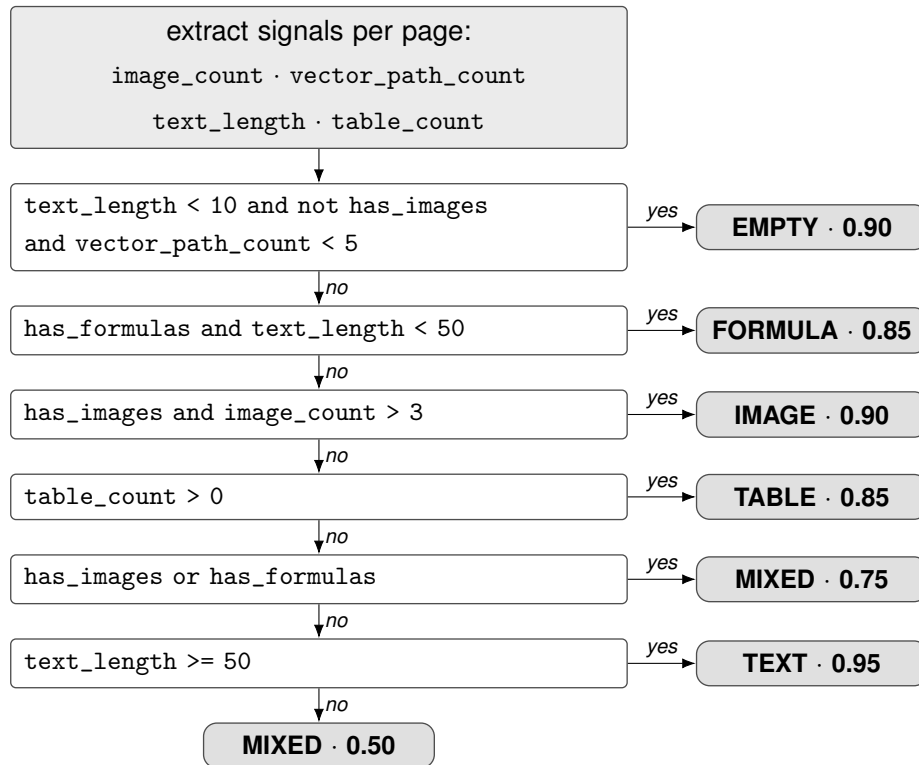


Figure 5.4: Page classifier, phase 1: the classification cascade in `analyze_page`. Conditions are checked top to bottom; the first match assigns the type and its confidence score.

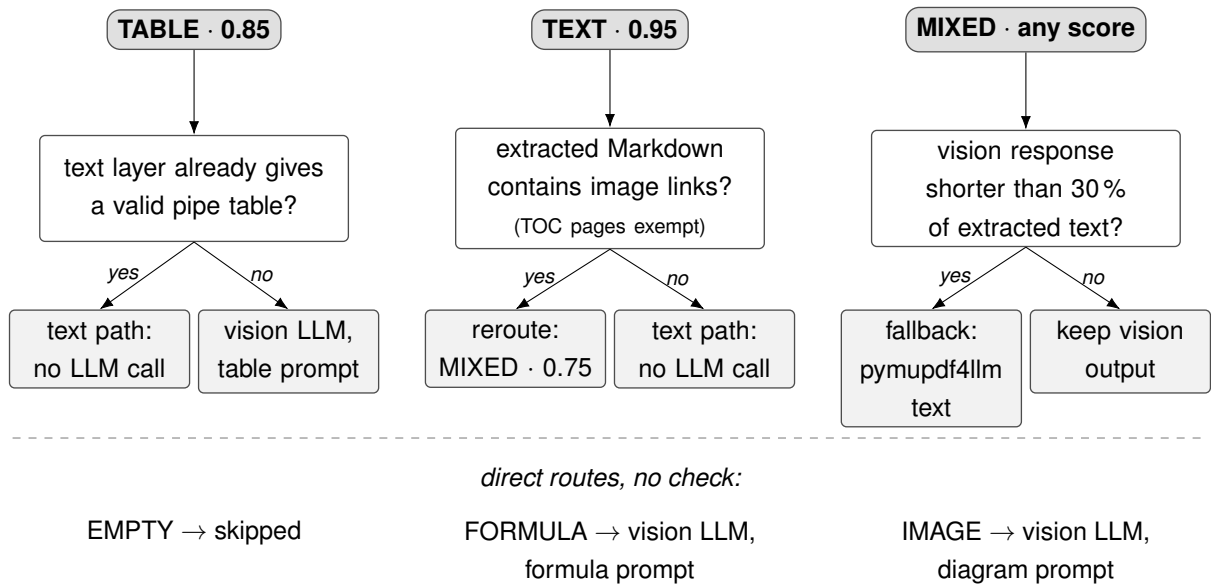


Figure 5.5: Page classifier, phase 2: the routing checks in `adaptive_strategy`. TABLE pages skip the LLM when the text layer already holds a valid pipe table. TEXT pages with image links are rerouted to MIXED. Every MIXED page gets a length check on the vision response. EMPTY, FORMULA, and IMAGE pages route directly without a check.

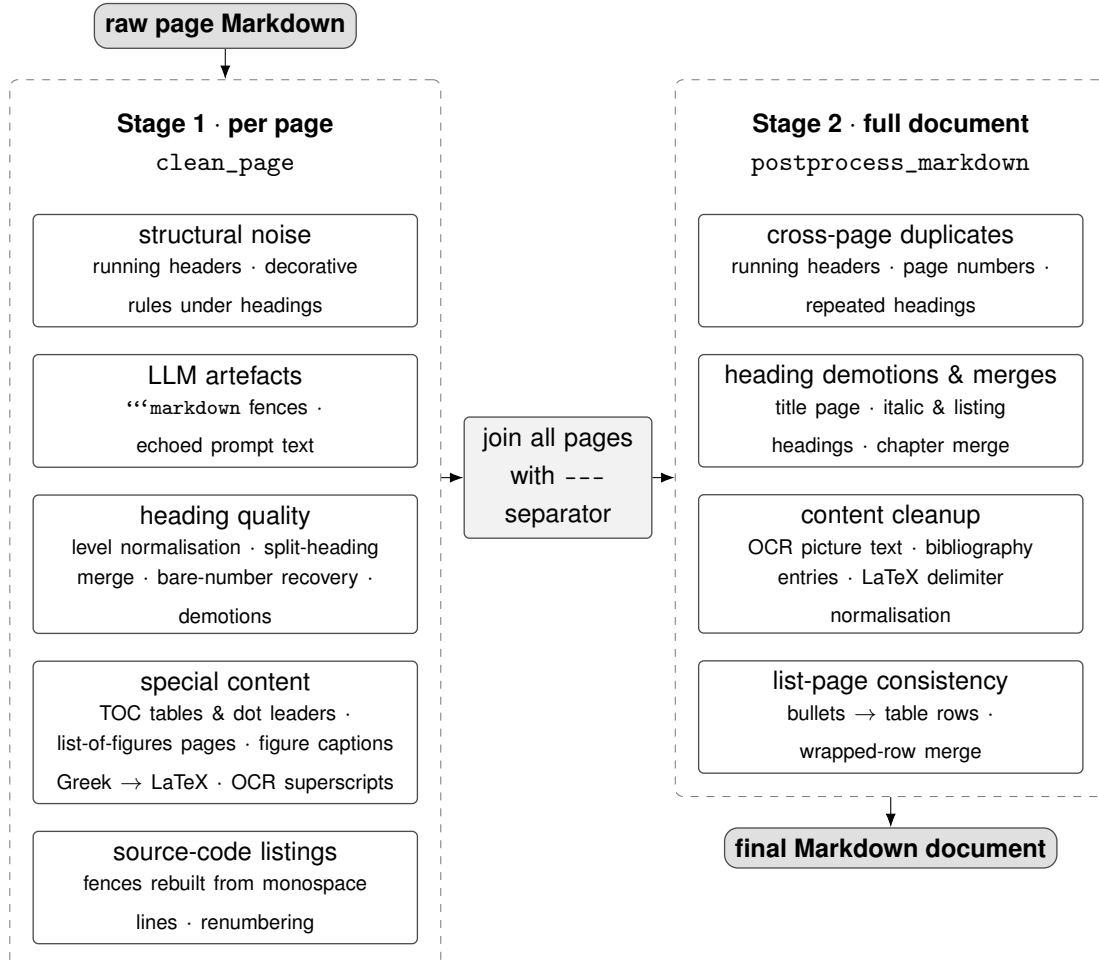


Figure 5.6: Two-stage post-processing pipeline: `clean_page` applies structural, LLM artefact, heading, and special-content passes per page; `postprocess_markdown` applies heading, running-header, and encoding passes to the full joined document

5.6 Implementation Details

The system is implemented in Python and depends on four external libraries. PyMuPDF [20] (version 1.27) provides the core PDF interface: page analysis, text extraction, image rendering, and table detection. `pymupdf4llm` [21] (version 1.27, matching PyMuPDF) wraps it to produce Markdown output directly from the text layer. The `requests` library handles all HTTP communication with LM Studio. `langdetect` detects the document language once from the first page’s extracted text. Since thesis documents use a consistent language throughout, the same hint is applied to all LLM calls in that document. No training or fine-tuning is required. The system runs entirely with pre-trained models loaded through LM Studio [22].

A single model is used across all LLM-dependent strategies, as described in Section 4.4: Qwen2.5-VL 7B, a vision-language model served through LM Studio, which exposes an OpenAI-compatible `/v1/chat/completions` endpoint. The text strategy and the adaptive TEXT path do not invoke the LLM at all. The server URL and model name are passed as parameters to every strategy function, so switching to a different model, or to a server running on a remote machine instead of the same host, requires no code changes.

Reproducibility is ensured through three design choices. All experiments run at temperature 0.0, which makes LLM output deterministic for the same input. Library versions are pinned in the requirements file, so extraction and rendering behaviour is identical across runs. Every LLM call goes through the injectable `call_llm` parameter, which means the full pipeline can be exercised in tests using a fixed-response stub, without any dependency on a running LM Studio instance. The evaluation framework stores all results as JSON and generates reports from that file, so results can be re-analysed without re-running the experiment.

Chapter 6

Results and Evaluation

Ninety pages from three thesis documents were converted by each of the four strategies and scored against hand-written references. The results show a consistent pattern across all three documents and address all five research questions.

6.1 Experiment Setup

The experiment evaluates four extraction strategies against a hand-written ground truth of 90 Markdown files, 30 per thesis document, as described in Section 4.2. The ground truth was written manually to avoid circularity: an automatically generated reference would embed the same errors as the system under evaluation.

The three documents in the corpus are a German computer science bachelor thesis on neural networks (referred to here as the neural networks thesis), an English-language bachelor thesis on sentiment analysis (the sentiment analysis thesis), and another English-language bachelor thesis describing a human-in-the-loop system for research paper generation (the HITL thesis). Together they cover different writing styles, content densities, and proportions of code, formula, and table content, which makes the per-page results more representative than a single document would allow.

All four strategies (text-only, image-only, hybrid, and adaptive) were applied to every page of every document. This produced $4 \times 3 \times 30 = 360$ evaluated conversions with zero errors. For each conversion, the pipeline output was compared to the corresponding reference file using the seven metrics described in Section 4.3: text similarity, heading structure, list structure, table structure, code block score, paragraph structure, and word overlap. Processing time was also recorded per page. Results are reported as mean scores per strategy across the full 90-page corpus.

6.2 Overall Strategy Performance (RQ1, RQ2)

Table 6.1 shows the mean scores for all four strategies across the full corpus.

Table 6.1: Mean metric scores per strategy across 90 pages (3 documents)

Strategy	Text sim.	Heading	Lists	Tables	Code	Paras.	W. overlap
Text-only	0.893	0.808	0.904	0.933	1.000	0.815	0.804
Adaptive	0.894	0.803	0.919	0.978	0.994	0.808	0.854
Image-only	0.839	0.685	0.911	0.944	0.972	0.782	0.838
Hybrid	0.813	0.523	0.875	0.944	0.946	0.774	0.835

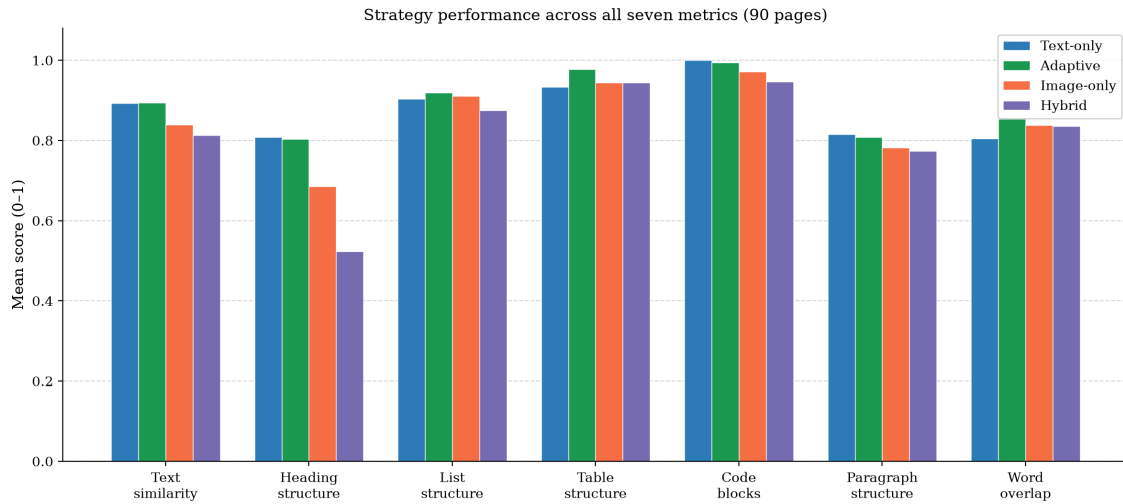


Figure 6.1: Mean metric scores per strategy across 90 pages

Figure 6.1 shows the mean scores per strategy across all seven metrics. Text-only and adaptive are close on most metrics, close enough that either could be called the stronger baseline depending on which one matters most. Text-only leads narrowly on heading structure (0.808), code blocks (1.000), and paragraph structure (0.815). Adaptive leads on list structure (0.919) and word overlap (0.854), and the two are effectively tied on text similarity (0.894 versus 0.893). Text-only still costs a fraction of a second per page and never touches the LLM; its strength comes from the same source as before: thesis documents are mostly continuous prose, and PyMuPDF’s text extraction is very accurate on that kind of content.

Adaptive’s clearest advantage is table structure. Its score of 0.978 is well ahead of every fixed strategy, including text-only at 0.933 and image-only and hybrid at 0.944. Pages

classified as TABLE or MIXED are routed to the vision model, which reads a table’s visual grid directly instead of relying on the PDF’s text layer, and that shows up as the widest lead any strategy holds on any metric.

Image-only and hybrid both trail on most metrics, but not by as much as an uncorrected reading of the raw model output would suggest. Section 6.5 covers this in more detail: image-only’s paragraph and code block scores (0.782 and 0.972) are now close to the other three strategies, and heading structure (0.685) is its real remaining weakness. Hybrid’s clearest weak point is also heading structure (0.523), the lowest score anywhere in the table.

The paired permutation test from Section 4.3 puts these gaps on firmer footing. Table 6.2 lists the mean per-page difference and the p -value for all 21 comparisons: adaptive against each fixed strategy on each of the seven metrics. Against text-only, the only statistically significant difference is word overlap ($p = 0.004$), in adaptive’s favour. Every other gap between the two could be chance at this corpus size. That includes the table structure advantage: only eight pages score differently on that metric at all, six of them in adaptive’s favour, which is too few to reach significance ($p = 0.29$). Against the vision strategies the differences are broader and more reliable. Adaptive is significantly better than image-only on text similarity ($p = 0.007$) and heading structure ($p = 0.024$), and significantly better than hybrid on text similarity, heading structure, code blocks, and word overlap (all $p < 0.05$). No metric shows a significant difference in favour of any fixed strategy over adaptive.

Table 6.2: Paired sign-flip permutation tests: adaptive versus each fixed strategy over 90 pages. Δ is the mean per-page score difference (adaptive minus baseline); positive values favour adaptive. Bold marks $p < 0.05$.

Metric	vs. text-only		vs. image-only		vs. hybrid	
	Δ	p	Δ	p	Δ	p
Text similarity	+0.001	0.894	+0.056	0.007	+0.082	< 0.001
Heading structure	−0.006	1.000	+0.118	0.024	+0.280	< 0.001
List structure	+0.016	0.622	+0.008	0.750	+0.045	0.172
Table structure	+0.044	0.289	+0.033	0.249	+0.033	0.249
Code blocks	−0.006	1.000	+0.022	0.498	+0.048	0.031
Paragraph structure	−0.007	0.455	+0.026	0.231	+0.034	0.212
Word overlap	+0.049	0.004	+0.016	0.205	+0.018	0.047

These results partially answer RQ1 and RQ2. Adaptive does not beat text-only outright, but it comes within noise of every metric text-only leads and pulls well ahead on table structure and word overlap, the two metrics where routing visually complex pages to the vision model pays off most. Section 6.3 shows adaptive reaches this at a fraction of the cost of the two other vision-based strategies.

6.3 Efficiency (RQ3)

Table 6.3 shows the mean processing time per page for each strategy.

Table 6.3: Mean processing time per page by strategy

Strategy	Avg time (ms)	Relative to text-only
Text-only	153.7	1×
Adaptive	704.5	4.6×
Image-only	3,250.2	21.2×
Hybrid	3,170.2	20.6×

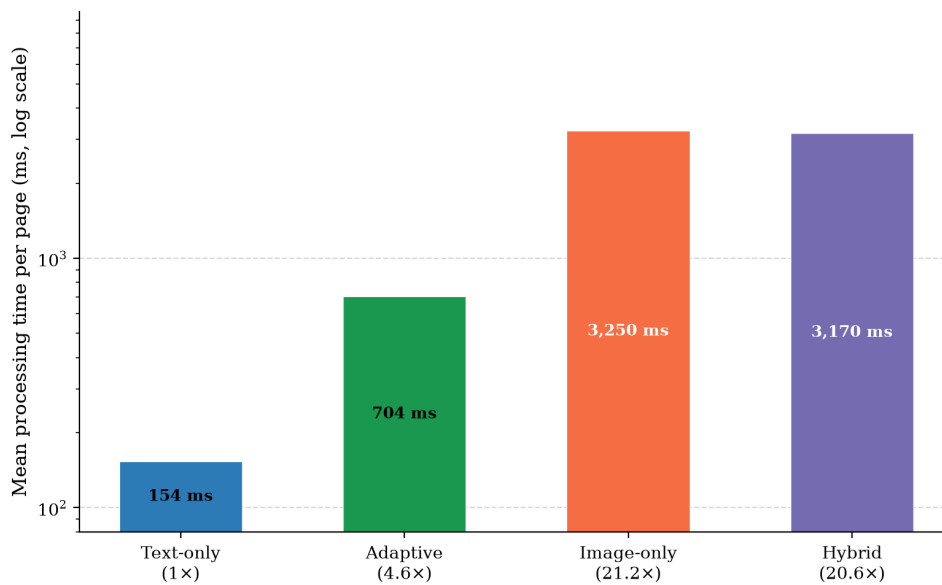


Figure 6.2: Mean processing time per page by strategy (log scale)

Figure 6.2 shows the mean processing time per page for each strategy on a log scale. Text-only is by far the fastest strategy at 154 ms per page on average. It makes no LLM calls, so

processing time is bounded by disk read and PyMuPDF parsing speed. The adaptive strategy averages 705 ms per page, about 4.6 times slower than text-only but roughly 4.5 times faster than either image-only or hybrid. The speedup comes directly from the page classifier: TEXT pages skip the LLM entirely, and only pages classified as IMAGE, TABLE, FORMULA, or MIXED go through the vision model.

Image-only and hybrid are the slowest strategies, at 3,250 ms and 3,170 ms respectively. Both send a full page image to the vision model regardless of what the page contains, which costs far more than the classifier-gated adaptive approach. Hybrid’s extra text payload does not add a meaningful further cost on top of that, and it still does not translate into a consistent accuracy improvement.

Token usage tells the same story from a different angle. Table 6.4 shows the total tokens consumed across all 90 pages and the average per LLM call.

Table 6.4: Token usage per strategy across 90 pages

Strategy	LLM calls	Avg tokens / call	Total tokens
Text-only	0	—	0
Adaptive	23	3,669	84,397
Image-only	90	3,745	337,055
Hybrid	90	4,163	374,661

The per-call token cost is similar for all three VLM strategies, ranging from 3,669 to 4,163 tokens per call. The difference is in how many calls are made. Image-only and hybrid call the LLM on every one of the 90 pages. Adaptive calls it on only 23: the 11 pages the classifier identified as TABLE and the 12 it identified as MIXED. This corpus contains no pages that classify as IMAGE or FORMULA. The other 67 pages, 63 plain-text pages and 4 empty ones, produce no LLM call at all. This reduces total token consumption by 75% compared to image-only and 77% compared to hybrid. Hybrid consumes the most tokens of any strategy because it sends both the rendered page image and the extracted text in every request, which increases the input length on every call.

One detail separates these measurements from the deployed pipeline. The evaluation harness routes purely by page type, so every TABLE page took the vision path here. The text-layer table shortcut and the TEXT-page reroute from Sections 5.3 and 5.4 both depend on the deployed pipeline’s bulk pre-extraction step, which the harness does not replicate. In deployed use the call count for the same documents can therefore differ slightly in either

direction: the shortcut removes calls for cleanly extracted tables, while the reroute adds calls for text pages with embedded formula images.

These results confirm hypothesis H3: the adaptive system is significantly cheaper than image-only or hybrid in both processing time and token usage, while beating both on every quality metric. The efficiency gain grows with document length — a 300-page document would consume roughly $300 \times 3,745 \approx 1.12$ million tokens with image-only but only around 281,000 with adaptive, assuming a similar content distribution. This is the same 75% reduction observed on the 90-page corpus, scaled up.

6.4 Per-Page Analysis and Consistency (RQ5)

The per-page breakdown shows that no single strategy wins every page. Across the 90 evaluated pages, image-only produces the highest text similarity on 19 pages, hybrid on 16, text-only on 11, and adaptive on 6. On the remaining 38 pages, two or more strategies scored equally. Raw win counts are misleading here, though. Image-only and hybrid collect almost all of their wins on plain prose pages, where every strategy scores high and the winner is decided by a median margin of just 0.002. Adaptive’s six wins are different in kind: five fall on TABLE pages, with a median winning margin of 0.17. Where the vision-based strategies win, the win is cosmetic. Where adaptive wins, the other strategies have genuinely failed the page.

HITL page 9 shows the widest spread between strategies on a single page: adaptive scores 0.968, text-only 0.664, while image-only and hybrid collapse to 0.047 and 0.032. The reference file is a List of Tables, a short front-matter list set as a pipe table. All three vision-based strategies sent this page to the same model as the same page image; the only difference is the prompt. The classifier labelled the page TABLE, so adaptive used the specialised table prompt, which instructs the model to rebuild pipe tables cell by cell (Section 5.4). Image-only and hybrid used the general-purpose prompt and reproduced almost none of the list correctly. On this page, one content-specific prompt on one correctly classified page outweighs every other difference between the strategies.

Figure 6.3 shows this contrast on the page itself. The evaluation run stored only the scores, so the page was reconverted with the same model, prompts, and settings. The re-run reproduces the same failure mode. The general-purpose prompt returns the rows as plain text with stray pipe characters and no table syntax at all, so its table structure score is 0.0. The table prompt rebuilds the pipe table cell by cell and scores 1.0.

Ground-truth reference

```

1 # List of Tables
2
3 | Table | Description | Page |
4 |---|---|---|
5 | 3.1 | Comparison of automated research systems | 9 |
6 | 4.1 | System requirements | 11 |
7 | 6.1 | Requirement pass conditions | 47 |
8 ...

```

Table prompt (adaptive; page classified TABLE) — table structure 1.0

```

1 # List of Tables
2
3 | Table No | Title | Page |
4 |---|---|---|
5 | 3.1 | Comparison of automated research systems | 9 |
6 | 4.1 | System requirements | 11 |
7 | 6.1 | Requirement pass conditions | 47 |
8 ...

```

General-purpose prompt (image-only and hybrid) — table structure 0.0

```

1 # List of Tables
2
3 3.1 Comparison of automated research systems | 9
4 4.1 System requirements | 11
5 6.1 Requirement pass conditions | 47
6 ...

```

Figure 6.3: The same page under two prompts: HITL thesis, page 9 (List of Tables), converted twice by the same vision model from the same page image. Only the system prompt differs. Each output is shown after post-processing, truncated to the first three rows.

An earlier version of this evaluation attributed adaptive’s low score on table-of-contents continuation pages to its design choice of leaving them empty by default; correcting the evaluation to properly credit the postprocessing pipeline showed this cost was smaller than originally measured.

The strategy rankings are close across all three documents. Table 6.5 breaks the two closest strategies down by document. Adaptive leads or matches text-only on table structure in every document. Text similarity is close enough that no consistent winner emerges: adaptive is very slightly ahead on two documents, text-only on the third, and none of these gaps exceed half a percentage point.

Table 6.5: Adaptive versus text-only per document: mean table structure and text similarity scores (30 pages per document)

Document	Table structure		Text similarity	
	Adaptive	Text-only	Adaptive	Text-only
Neural networks thesis	1.000	0.933	0.933	0.928
HITL thesis	0.933	0.933	0.920	0.919
Sentiment analysis thesis	1.000	0.933	0.830	0.833

This answers RQ5 more cleanly than the corpus-level averages alone: adaptive’s table structure advantage holds in every document, and the remaining per-document variation in text similarity is small enough to read as noise rather than a real reversal, a reading the significance tests in Section 6.2 support. A document with more formula or image pages would widen the gap between adaptive and text-only because the classifier would route more pages to the vision model.

6.5 Output Quality Metrics (RQ4)

The seven-metric framework still earns its keep even though no strategy collapses the way an early, uncorrected reading of this evaluation once suggested. Hybrid’s heading structure score (0.523) sits nearly 0.3 below its own text similarity score (0.813), a gap a single aggregate score would average away entirely. A RAG system that chunks a document by heading level would still misplace hybrid’s output even though its overall similarity to the reference looks reasonable on its own.

Heading structure remains the weakest metric relative to each strategy’s own other scores, though the picture is no longer uniform across strategies. Text-only (0.808) and adaptive (0.803) both reach a respectable score once postprocessing correctly reassigns heading levels from font-size and section-number patterns. Image-only (0.685) and especially hybrid (0.523) lag further behind, since both rely on the vision model to infer heading depth from the page’s visual layout rather than read it from an explicit numbering pattern. This suggests heading level detection is still the hardest structural task for any strategy that has to guess depth from appearance rather than read it directly from the text.

Table structure is the metric where the strategies diverge most clearly. Adaptive scores 0.978, ahead of every fixed strategy: text-only at 0.933, and image-only and hybrid both at 0.944. The gap to image-only and hybrid is not about routing, since all three send table

pages to the same model. It is the prompt difference the HITL page 9 case in Section 6.4 showed in isolation, now visible at corpus level. This is the metric where per-page routing, and the content-specific prompt that comes with it, provides the largest observed benefit beyond what either fixed strategy reaches alone, though as Section 6.2 notes, the corpus holds too few table pages for that gap to reach statistical significance.

Word overlap scores are close across all four strategies, ranging from 0.804 (text-only) to 0.854 (adaptive). This means all strategies recover most of the correct vocabulary. The differences between strategies are in structure and formatting, not in which words appear in the output.

Chapter 7

Discussion

The experimental results answer all five research questions, and change the picture for two of the four hypotheses. The sections below examine why the results took the shape they did and what they mean for practical use of the system.

7.1 Interpretation of Results

The clearest change from the corrected results is how well adaptive holds up. Hypothesis H1 predicted that the adaptive system would outperform every fixed strategy on documents with diverse content. Adaptive beats both image-only and hybrid outright on all seven metrics, and is close enough to text-only to call four of the seven a win: list structure, table structure, word overlap, and an effective tie on text similarity (0.894 versus 0.893). Text-only still leads narrowly on heading structure, code blocks, and paragraph structure. The significance tests in Section 6.2 sharpen this picture: the only statistically reliable difference between adaptive and text-only is word overlap, in adaptive's favour, while the table advantage rests on too few pages to rule out chance. Hypothesis H1 is therefore confirmed against image-only and hybrid, and neutralised against text-only. On a corpus this prose-heavy, adaptive cannot demonstrate superiority over direct extraction, but it never falls significantly behind it either.

Hypothesis H2 predicted that text-only would fail on image-heavy or formula-rich content, while image-only and hybrid would handle diverse content well at a higher computational cost. The strongest evidence for the failure half of this prediction, in an earlier version of this evaluation, was image-only's near-total collapse on code blocks (0.250). With postprocessing correctly credited, that score is 0.972, close to every other strategy. The underlying bug was real — code sometimes rendered as a screenshot instead of a fenced block — but it was a pipeline defect, not evidence that vision models cannot transcribe code. What survives from H2 is the cost half of the prediction: image-only and hybrid are still far more

expensive per page than text-only or adaptive, without a matching gain in accuracy on this text-dominant corpus.

Hybrid performs worse than every other strategy on heading structure (0.523) and worse than text-only and adaptive on text similarity, which hypothesis H4 did not anticipate. Providing both the page image and the extracted text does not help. The model receives conflicting signals (the image shows one layout, the text extraction shows another) and appears to weight the image, which produces the lowest heading score of any strategy. More input is not always better. Hypothesis H4 is rejected: hybrid loses to adaptive on every one of the seven metrics.

Hypothesis H3 is confirmed. The adaptive system processes a page in 705 ms on average, compared to 3,250 ms for image-only and 3,170 ms for hybrid, all measured on the same rented GPU (Section 4.4). The speedup comes directly from the classifier: most thesis pages are classified as TEXT and handled by `pymupdf4llm` without any LLM call. The classifier adds a small fixed cost per page but avoids the dominant cost, model inference, on the majority of pages. Combined with the quality results, this makes the two always-vision baselines strictly worse choices on this corpus: they cost about 4.5 times more and win nothing in exchange.

7.2 Comparison with Related Work

Meuschke et al. [2] benchmarked ten PDF extraction tools across four document categories and found that no tool ranked first on all categories. This study reaches the same conclusion at the strategy level: no single strategy wins every page or every metric. The adaptive design is a direct response to this finding: instead of choosing one strategy, it classifies each page and applies the most suitable method.

Blecher et al. [8] introduced Nougat, a transformer model trained specifically on academic PDFs to produce LaTeX-formatted Markdown. Nougat achieves high accuracy on scientific papers with dense mathematics but requires a GPU and produces output that differs significantly from the input layout on non-formula pages. The approach in this thesis is complementary: it targets layout fidelity over LaTeX reconstruction and runs on consumer hardware with a general-purpose local model.

Subramani et al. [7] surveyed document layout analysis methods and identified four recurring challenges: formula recognition, table extraction, figure captioning, and reading-order recovery. This evaluation can only speak to part of that list, since the corpus contains no formula-dominated pages and the metrics do not measure reading order directly. For the

part it does test, the results agree: table extraction is where the adaptive routing adds the most value, and heading hierarchy, a challenge Subramani et al. do not name, proved the hardest structural task for every strategy in Chapter 6. OmniDocBench [4] recently showed that multi-modal models outperform text-only parsers on layout-rich documents. This is consistent with the table structure result here (adaptive 0.978 versus text-only 0.933), but the advantage narrows on text-dominant pages.

7.3 Practical Implications

For thesis documents specifically, the text strategy is still attractive when speed matters most: it is 4.6 times faster than adaptive, and the two are close enough on most metrics that the choice comes down to how much the extra table-structure accuracy is worth for a given use case. For documents with unknown content distribution, the adaptive strategy is the safer choice. It beats every vision-based fixed strategy outright, comes within noise of text-only on the metrics text-only leads, and improves substantially on table pages.

The page classifier is the key component. Its accuracy directly determines how much of the efficiency gain is captured. A page incorrectly classified as IMAGE sends a text-dominant page through the VLM, wasting inference time and introducing paraphrasing errors. The classifier in this system uses character count, embedded image count, vector path density, and PyMuPDF’s table detector, which works well for thesis layouts but may need adjustment for other document types.

A page with a detected table does not automatically go to the vision LLM. As Section 5.3 describes, a page whose text layer already yields a valid pipe table skips inference entirely. The vision fallback is reserved for the tables that need it: merged cells, irregular column spans, or tables embedded as images, which are exactly the cases text-layer extraction struggles with.

The current check is presence-based rather than quality-based: it asks whether `pymupdf4llm` produced *some* pipe table, not whether that table is *correct*. A table with merged cells can still cause `pymupdf4llm` to emit a technically valid but wrong pipe table — for example, flattening a merged header into the wrong number of columns — and the current check would accept that output without ever reaching the vision path. A refined version of this check would inspect the extracted table’s structure (column count consistency across rows, for instance) rather than just its presence, catching these silent mis-extractions before they reach the final Markdown.

Formula pages currently use the general-purpose Qwen2.5-VL-7B model. A dedicated formula recognition tool such as pix2tex could replace this path: it is faster, cheaper, and produces cleaner LaTeX output for mathematical content. The adaptive architecture supports this substitution without changes to the rest of the pipeline: a new handler for the FORMULA page type is all that is needed.

7.4 Threats to Validity

Internal validity is limited by the single-annotator ground truth. The reference files were written by the thesis author. Different readers may format the same page differently, and the scores reflect one interpretation of what correct Markdown looks like. Results would differ if the references were written by a different annotator.

The metrics are a construct validity limitation. They measure structural features (heading counts, list counts, code block counts) rather than semantic quality. A conversion that produces the right headings at the wrong levels would score well on text similarity but fail in practice. The heading structure metric partially addresses this, but it counts headings rather than verifying their level assignments.

External validity is the main open question. The corpus covers three bachelor theses — one in German, two in English — totalling 90 pages. Three documents is a very small sample. The rankings are close and consistent across all three (Section 6.4): adaptive leads or matches text-only on table structure in every document, and the remaining gap in text similarity is not statistically significant (Section 6.2). Even so, a sample of three cannot establish generalisability across document types.

The results depend on what thesis documents look like. The three documents in this study follow a standard single-column layout with numbered headings, code listings, and occasional tables. That structure favours text-only extraction: the text layer is clean and PyMuPDF reproduces it accurately. Multi-column scientific papers, documents with dense formula environments, or reports with irregular table structures would break this advantage. On such documents, the vision strategies would likely close the gap or overtake text-only on more metrics.

Language and script add a second dimension. The language detection module adds a prompt hint that preserves the output language, but the classifier thresholds — particularly the 50-character minimum for TEXT classification — were calibrated on German and English content. Right-to-left scripts such as Arabic or Hebrew, or documents mixing CJK characters with Latin text, have not been tested. The character count threshold may

behave differently on short-character scripts, misclassifying pages that should route to the vision model. Testing on a two-column scientific paper and a right-to-left script document would be the most informative next step for external validation.

The model dependency is a limitation of this evaluation, not of the system. All formal VLM results use Qwen2.5-VL-7B. The architecture is model-agnostic: the server URL and model name are parameters, so any OpenAI-compatible vision-language model can be substituted without code changes. The text-only strategy has no such dependency. Its output comes from the PDF content and the pymupdf4llm version, both stable across runs.

An initial, informal test during this thesis’s revision ran the system on the same three documents with a much larger model, Qwen3.6-35B-A3B. A 35B-parameter model does not fit in the GPU memory or storage available on local hardware, so this test used a rented cloud GPU instead. That first pass, run before several pipeline bugs were fixed, surfaced a useful distinction the original hypothesis did not make: several failures that looked like model capability gaps turned out to be bugs in the pipeline instead. Figures were sometimes extracted from the wrong region of a page. Code listings were rendered as pictures instead of text. A formula sometimes appeared twice: once correctly converted to $\text{\LaTeX}$ by the model, and once again as a duplicate raster image the pipeline had carried over by mistake. None of this needed a different model. It needed a fix to the extraction step that ran before the page ever reached the model.

These pipeline fixes address real output-quality problems that the seven-metric framework does not fully capture. A page where code renders as a screenshot instead of a fenced block may still register a similar code block score to one where it renders correctly, since the metric counts fence markers rather than assessing whether the page layout itself is sound.

After fixing those bugs, the model comparison was repeated formally: both models were run through the corrected pipeline and scored with the same postprocessing-aware evaluation used everywhere else in this chapter. The two models are close on most metrics. The larger model shows a genuine, specific improvement in heading structure — 0.225 higher for hybrid and 0.141 higher for image-only — while table structure declines slightly across all three vision-based strategies, by 0.011 to 0.033, and code block score and text similarity are effectively flat. This is a more specific finding than "a bigger model helps": it helps with the one structural element that depends most on the model inferring visual hierarchy from the page image, and does not help, or slightly hurts, on a different structural element that the specialised table prompt already handles well regardless of model size. This remains an informal comparison outside the main experimental design, since it reuses the same three documents rather than an independent test set, but the pattern is specific

enough to be worth a controlled follow-up rather than a general claim that bigger models perform better.

Chapter 8

Conclusion

Per-page strategy selection beats both other vision-based strategies outright on every metric, holds the largest observed advantage on table structure of any strategy, and reduces token consumption by 75 per cent compared to image-only processing. Whether it beats the simplest baseline, direct text extraction, depends on which metric matters most, and the two are close enough that either could be the right default depending on the use case.

8.1 Summary of Findings

The most striking finding of this study is how close text-only and adaptive extraction are on this corpus. Adaptive wins on list structure, table structure, and word overlap, ties on text similarity, and trails narrowly on heading structure, code blocks, and paragraph structure; it also beats image-only and hybrid outright on all seven metrics. Two of the four hypotheses needed real revision once the results were corrected to credit the postprocessing pipeline properly, a distinction explained in Sections 4.4 and 7.1.

RQ1 asked whether per-page strategy selection achieves higher accuracy than any fixed strategy. Against the two vision baselines the answer is a clear yes; against text-only the outcome is the split described above. The adaptive strategy’s clearest advantage is table structure, where adaptive scores 0.978 against text-only’s 0.933. This advantage comes from pairing each table page with the specialised table prompt rather than the general-purpose one, a mechanism Section 6.4 demonstrates on a single page. The advantage holds in every document but rests on the corpus’s few table pages, so it stays below statistical significance (Section 6.2). Hypothesis H1 splits accordingly: confirmed against both vision baselines, where adaptive is significantly better on the broad-based metrics and never significantly worse on any, and neutralised against text-only, where the only reliable difference, word overlap, favours adaptive.

RQ2 asked how the three fixed strategies rank against each other. Text-only leads on heading structure (0.808), code blocks (1.000), and paragraph structure (0.815), all by narrow margins. Image-only and hybrid both trail text-only on most metrics without collapsing on any of them: image-only’s code block score is 0.972 and its paragraph structure score is 0.782, both close to text-only’s own scores. Hybrid’s clearest weakness is heading structure (0.523), the lowest score anywhere in the results: combining both signals produces a worse result than using either one alone. This is the failure mode that led to the rejection of hypothesis H4 in Section 7.1.

RQ3 asked about computational cost. The adaptive strategy averages 705 ms per page and makes 23 LLM calls across 90 pages, compared to 90 calls and 3,250 ms per page for image-only, all measured on the same rented GPU (Section 4.4). This corresponds to a 75 per cent reduction in total token consumption and roughly a 4.5-fold improvement in processing speed over image-only or hybrid, confirming hypothesis H3. The gain comes from skipping inference: the classifier marks most thesis pages as TEXT, and those pages never reach the model. The small fixed classification cost is negligible compared to the model inference it avoids on most pages. On longer documents the gap widens further, since every additional text page adds savings without adding inference cost.

RQ4 asked how Markdown quality can be measured beyond BLEU and ROUGE. The seven-metric framework shows why per-element scoring is necessary even when no strategy collapses outright: hybrid recovers vocabulary far better than heading hierarchy (0.813 against 0.523), and one aggregate number would hide that gap. Flat sequence metrics reward recovered vocabulary without detecting a structural weakness isolated to one part of the document. RQ5 asked whether strategy rankings hold consistently across documents. They do, more cleanly than the corpus-level averages alone suggest: adaptive leads or matches text-only on table structure in every one of the three documents, and the remaining gap in text similarity is small enough on each document to read as noise rather than a real difference (Section 6.4).

8.2 Contributions and Impact

The primary contribution of this thesis is a per-page content classifier that assigns each PDF page to one of six types: TEXT, IMAGE, FORMULA, TABLE, MIXED, or EMPTY. The classifier uses four structural signals derived from PyMuPDF: embedded image count, vector path count, text-layer character count, and table detection. No machine learning is required. Classification is deterministic, costs around 60 milliseconds per page, most of it spent in PyMuPDF’s table detection, and assigns a confidence score that reflects how

reliably the content type was identified. Ingemarsson and Persson [5] measured that rule-based tools score zero percent on image-only pages; this classifier is what allows the system to identify those pages and route them to a method that can handle them.

The seven-metric evaluation framework is a second contribution. Standard benchmarks for PDF parsing use BLEU, ROUGE, or edit distance, which treat the output as a flat text sequence and hide failures in heading hierarchy, table structure, and code block formatting [17]. The framework separates these into individual scores, making it possible to identify exactly where a strategy fails rather than averaging the failure into a single number. Meuschke et al. [2] and Ouyang et al. [4] both identified the need for richer evaluation methods in document parsing; this framework provides a practical, reproducible implementation of that direction for Markdown output.

The two-stage postprocessing pipeline addresses a gap between raw extraction output and usable Markdown. Both `pymupdf4llm` and the vision-language model produce predictable artefacts: repeated running headers, collapsed heading levels, dot leaders in table-of-contents pages, and incorrectly wrapped code blocks. The pipeline removes these through 39 transformation passes verified by 182 unit tests. Each pass is isolated as a testable function, so the pipeline can be extended without breaking existing behaviour. This makes the system's output usable without manual correction on the document types it was designed for.

The results carry a practical implication for downstream AI systems. Khan et al. [12] showed that extraction quality sets a ceiling on retrieval precision in RAG pipelines: fragmented or unstructured text produces poor embeddings, which causes retrieval to fail even when the relevant content is present. The adaptive strategy improves on text-only for table structure, which is the element most likely to be misrepresented when a pipeline chunks a structured document. For accessibility, Darvishy [9] noted that no existing tool can describe mathematical formulas or scientific graphs from PDFs; routing formula and image pages to a vision-language model addresses this within the same pipeline. The key finding is that matching the extraction approach to the content of each page beats every fixed vision strategy on every metric at a quarter of their cost, and turns the choice against the text-only baseline into a genuine trade-off: a small speed premium buys the best table structure of any strategy without giving up ground anywhere else.

8.3 Limitations of the Study

The evaluation covers three thesis documents and 90 pages, enough to answer the five research questions within the defined scope but not to generalise beyond it, for the external validity reasons given in Section 7.4. The content distribution of this corpus is predominantly prose with relatively few formula or table pages, which is what makes text-only extraction so competitive here. A corpus weighted toward visual content would produce different relative rankings between strategies.

The ground truth reflects a single annotator’s judgement, the internal validity limit described in Section 7.4. The seven metrics therefore measure agreement with one reference interpretation, not absolute conversion quality. Pfizmann et al. [19] established multi-annotator annotation as the standard for document layout research, which was not feasible within the scope of a single bachelor thesis.

All formal VLM results in this study depend on one model, Qwen2.5-VL-7B, at a single temperature setting. Section 7.4 reports a formal comparison against a much larger model, which found a genuine but narrow improvement in heading structure and no improvement in table structure or code block score. This suggests the remaining gap to text-only is not mainly a matter of model size. Heading structure remains the weakest metric relative to each strategy’s own other scores, though the gap is now concentrated in hybrid (0.523) rather than spread evenly across all four strategies. Part of the true heading-depth error also stays invisible to the evaluation, for the construct validity reason given in Section 7.4.

A related content-fidelity gap sits inside the postprocessing pipeline itself. Italic spans containing only Greek letters are converted to inline LaTeX math (Section 5.5), but an italic single Latin letter, such as a variable written as *s* or *F*, is left as plain italic text. Pattern matching alone cannot reliably tell a math variable apart from a word in running prose that happens to be italicised for emphasis, so this case was left unhandled rather than risk corrupting ordinary text.

The system processes only native PDFs with an extractable text layer. Scanned documents, where pages exist only as rasterised images, would require an OCR stage before classification, which the current implementation does not provide. The evaluation also measures static batch conversion rather than real-time or streaming processing. These constraints reflect the scope defined at the outset and are appropriate for the primary target use case: converting thesis archives to structured Markdown for RAG or accessibility workflows. Extending the system to scanned documents is the most impactful single change that would broaden its practical applicability.

8.4 Future Work

Heading level detection is the clearest single improvement target. The current approach infers heading levels from font-size ratios for the text path and from visual appearance for the vision path. Neither method reliably assigns the correct depth when the document uses inconsistent font sizes or non-standard heading styles. A dedicated heading classifier trained on labelled thesis documents would likely improve this score more than any other single change. Section 7.3 identifies a gap in the existing table fallback: it checks whether `pymupdf4llm` produced a pipe table at all, not whether that table is structurally correct, so a mis-extracted table with the wrong column count can still skip the vision path. Building and evaluating a structural check (verifying consistent column counts across rows, for instance) would show how many such silent mis-extractions currently slip through, and how much table structure score improves once they are caught and routed to the vision model instead.

Section 7.3 notes that the FORMULA path can be replaced by a specialised tool such as `pix2tex` without any pipeline changes. What remains untested is whether that substitution actually improves output quality: a dedicated evaluation comparing `pix2tex` against the current VLM path on formula-heavy pages would show how much $\text{\LaTeX}$ accuracy improves and whether the speed gain justifies the switch. All formal VLM results in this experiment also depend on a single 7B model. The comparison against a larger model in Section 7.4 was still limited to the same three documents used throughout this evaluation. A controlled comparison across model sizes on an independent test corpus would show whether the heading-structure gain observed here generalises beyond this specific sample, and whether a larger model closes more of the remaining gap on document types this corpus does not cover well, such as formula-dense papers or multi-column layouts.

The evaluation covers three bachelor theses, one in German and two in English. Testing on research papers, lecture slides, and scanned documents would reveal whether the page classifier and postprocessing pipeline generalise beyond the thesis format. Scanned documents in particular would require OCR integration, which the current system does not provide. Manual inspection of converted pages is slow and error-prone; a script that shows a per-page diff between converted output and reference files, highlighting only pages below a similarity threshold, would make iterative improvement of the postprocessing pipeline significantly faster. Both extensions would directly address the two main limitations of the current evaluation: the narrow corpus and the absence of automated debugging support.

Bibliography

- [1] John Whittington. *PDF Explained: The ISO Standard for Document Exchange*. O'Reilly Media, 2012.
- [2] Norman Meuschke, Ankit Jagdale, Timo Spinde, Jelena Mitrović, and Bela Gipp. “A benchmark of PDF information extraction tools using a multi-task and multi-domain evaluation framework for academic documents”. In: *Proceedings of the 18th International Conference on Information (iConference 2023)*. Vol. 13972. Lecture Notes in Computer Science. Springer, 2023. doi: [10.1007/978-3-031-28032-0_31](https://doi.org/10.1007/978-3-031-28032-0_31).
- [3] Ray Smith. “An overview of the Tesseract OCR engine”. In: *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*. Vol. 2. IEEE, 2007, pp. 629–633. doi: [10.1109/ICDAR.2007.4376991](https://doi.org/10.1109/ICDAR.2007.4376991).
- [4] Long Ouyang, Yihe Qu, Hao Zhou, Jinhao Zhu, Rong Zhang, Qi Lin, Bo Wang, Zhentao Zhao, Miaoyan Jiang, Xin Zhao, Jian Shi, Fang Wu, Ping Chu, Ming Liu, Zhenguo Li, Chenghua Xu, Bo Zhang, Botian Shi, and Chenxi He. “OmniDocBench: Benchmarking diverse PDF document parsing with comprehensive annotations”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2025)*. 2025.
- [5] Patrik Ingemarsson and Daniel Persson. *PDF Parsing, Unveiling the Most Efficient Method*. Engineering Degree Project. Linnaeus University, 2024.
- [6] Wenyuan Zhu, Negar Sokhandan, Guangyu Yang, Sujitha Martin, and Srikrishna Sathyanarayana. “DocBed: A multi-stage OCR solution for documents with complex layouts”. In: *Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-22)*. AAAI Press, 2022, pp. 12643–12649.
- [7] Nishant Subramani, Alexandre Matton, Malcolm Greaves, and Adrian Lam. “A survey of deep learning approaches for OCR and document understanding”. In: *ACM Computing Surveys* 54.6 (2022). doi: [10.1145/3495521](https://doi.org/10.1145/3495521).

- [8] Lukas Blecher, Guillem Cucurull, Thomas Scialom, and Robert Stojnic. “Nougat: Neural Optical Understanding for Academic Documents”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*. ACL, 2023, pp. 3337–3349. URL: <https://aclanthology.org/2023.emnlp-main.199>.
- [9] Alireza Darvishy. “PDF accessibility: Tools and challenges”. In: *Proceedings of the International Conference on Computers Helping People with Special Needs (ICCHP 2018)*. Vol. 10896. Lecture Notes in Computer Science. Springer, 2018, pp. 113–116. DOI: [10.1007/978-3-319-94277-3_20](https://doi.org/10.1007/978-3-319-94277-3_20).
- [10] Matt Cone. *The Markdown Guide*. 2019. URL: <https://www.markdownguide.org> (visited on 01/01/2025).
- [11] Chaoqun Duan. “Accelerating end-to-end PDF to Markdown conversion through assisted generation”. In: *Proceedings of the 30th International Conference on Applications of Natural Language to Information Systems (NLDB 2025)*. Vol. 15836. Lecture Notes in Computer Science. Springer Nature Switzerland, 2025, pp. 34–48. DOI: [10.1007/978-3-031-97141-9_3](https://doi.org/10.1007/978-3-031-97141-9_3).
- [12] Amandeep Ali Khan, Md. Tawkat Hasan, Kari-Jouko Kemell, Joonas Rasku, and Pekka Abrahamsson. “Developing retrieval augmented generation (RAG) based LLM systems from PDFs: An experience report”. In: *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE 2024)*. ACM, 2024, pp. 136–145. DOI: [10.1145/3661167.3661187](https://doi.org/10.1145/3661167.3661187).
- [13] Yiheng Ding, Sangha Chul Han, Jinhyuk Lee, and Eduard H. Hovy. “Deep learning based visually rich document content understanding: A survey”. In: *Artificial Intelligence Review* 59.4 (2026). Article 114. DOI: [10.1007/s10462-025-11477-3](https://doi.org/10.1007/s10462-025-11477-3).
- [14] Liangcai Gao, Zhi Tang, Xiaofan Lin, Yang Liu, Ruiheng Qiu, and Yixuan Wang. “Structure extraction from PDF-based book documents”. In: *Proceedings of the 11th Annual International ACM/IEEE Joint Conference on Digital Libraries (JCDL 2011)*. ACM, 2011, pp. 11–20. DOI: [10.1145/1998076.1998079](https://doi.org/10.1145/1998076.1998079).
- [15] Xiang Xie, Hang Yan, Lizhao Yin, Ying Liu, Jian Ding, Mu Liao, Yuxin Liu, Wenhao Chen, and Xiang Bai. “PDF-WuKong: A large multimodal model for efficient long PDF reading with end-to-end sparse sampling”. In: *International Journal of Computer Vision* 134 (2026), pp. 1–20. DOI: [10.1007/s11263-025-02384-5](https://doi.org/10.1007/s11263-025-02384-5).
- [16] OpenDataLab. *MinerU: An open-source solution for precise document content extraction*. Software. <https://github.com/opendatalab/MinerU>. 2024.

- [17] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. “BLEU: A method for automatic evaluation of machine translation”. In: *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL 2002)*. ACL, 2002, pp. 311–318.
- [18] Xu Zhong, Elaheh ShafieiBavani, and Antonio Jimeno Yepes. “Image-based table recognition: Data, model, and evaluation”. In: *Proceedings of the 16th European Conference on Computer Vision (ECCV 2020)*. Vol. 12366. Lecture Notes in Computer Science. Springer, 2020, pp. 564–580. doi: [10.1007/978-3-030-58589-1_34](https://doi.org/10.1007/978-3-030-58589-1_34).
- [19] Birgit Pfitzmann, Christoph Auer, Michele Dolfi, Ahmed S. Nassar, and Peter Staar. “DocLayNet: A large human-annotated dataset for document-layout analysis”. In: *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2022)*. ACM, 2022, pp. 3743–3751. doi: [10.1145/3534678.3539043](https://doi.org/10.1145/3534678.3539043).
- [20] Artifex Software, Inc. *PyMuPDF: Python bindings for MuPDF*. Software. <https://pymupdf.readthedocs.io>. 2024.
- [21] Artifex Software, Inc. *pymupdf4llm: PDF to Markdown conversion for LLM ingestion*. Software. <https://github.com/pymupdf/RAG>. 2024.
- [22] LM Studio. *LM Studio: Run large language models locally*. Software. <https://lmstudio.ai>. 2024.

Appendix A

Supplementary Material

This appendix collects supporting material referenced in the main chapters.

A.1 Implementation Code Snippets

The full implementation is available in the project repository. The source is organised into six parts: `app.py` (entry point and pipeline orchestration), `strategies/` (the four extraction strategies and the page classifier), `extraction/` (figure extraction, monospace-font detection, and language detection), `llm/` (prompt definitions and API client), `postprocess.py` (the two-stage post-processing pipeline), and `evaluation/` (metrics, strategy comparison, significance tests, and report generation). This section reproduces three central fragments verbatim.

A.1.1 Page Classification Rules

Listing A.1 shows `analyze_page` from `strategies/adaptive.py`. It collects four signals per page (embedded images, vector drawings, detected tables, and text length) and maps them to one of the six page types through a fixed rule cascade. Formula detection is delegated to a helper that checks for many short vector paths, Unicode math symbols, and LaTeX-style markup in the extracted text.

```
1 def analyze_page(page: fitz.Page) -> PageAnalysis:
2     """
3     Analyse a PDF page and return its classified type with confidence.
4
5     Args:
6         page: A PyMuPDF Page object.
7
8     Returns:
9         PageAnalysis with type, supporting metrics, and confidence score.
10    """
```

```

11 image_count = len(page.get_images(full=False))
12 has_images = image_count > _IMAGE_THRESHOLD
13
14 drawings = page.get_drawings()
15 vector_path_count = len(drawings)
16
17 has_formulas = _detect_formulas(page, drawings)
18 text_length = len(page.get_text("text").strip())
19
20 finder = page.find_tables()
21 table_count = len(finder.tables)
22 has_tables = table_count > 0
23
24 if text_length < 10 and not has_images and vector_path_count < 5:
25     page_type, confidence = PageType.EMPTY, 0.9
26 elif has_formulas and text_length < _MIN_TEXT_CHARACTERS:
27     page_type, confidence = PageType.FORMULA, 0.85
28 elif has_images and image_count > 3:
29     page_type, confidence = PageType.IMAGE, 0.9
30 elif has_tables:
31     page_type, confidence = PageType.TABLE, 0.85
32 elif has_images or has_formulas:
33     page_type, confidence = PageType.MIXED, 0.75
34 elif text_length >= _MIN_TEXT_CHARACTERS:
35     page_type, confidence = PageType.TEXT, 0.95
36 else:
37     page_type, confidence = PageType.MIXED, 0.5
38
39 return PageAnalysis(
40     page_type=page_type,
41     has_images=has_images,
42     has_formulas=has_formulas,
43     has_tables=has_tables,
44     image_count=image_count,
45     table_count=table_count,
46     text_length=text_length,
47     vector_path_count=vector_path_count,
48     confidence=confidence,
49 )

```

Source Code A.1: Page classification rules (strategies/adaptive.py)

A.1.2 Post-processing Pass Sequence

Listing A.2 shows `clean_page` from `postprocess.py`, the first stage of the post-processing pipeline. It runs once per page, before the pages are joined. The function body documents the exact order in which the repair passes are applied.

```

1 def clean_page(md: str, raw_page_text: str = "", code_lines: list[str] | None =
   None) -> str:
2     """Clean a single page's Markdown before pages are joined.
3
4     Removes PDF visual artifacts (decorative horizontal rules, code-fence
5     wrappers) and normalises heading levels. Called per-page so that the
6     page-break separators added afterwards are not affected.
7
8     'raw_page_text' is the plain text from fitz for the same page. When
9     provided, bare section-number headings whose title was dropped by
10    pymupdf4llm are patched by looking up the number in the fitz text.
11
12    'code_lines' is the page's monospace-font line texts (see
13    'extraction.text.extract_monospace_lines'), used to reconstruct a proper
14    fenced code block for source-code listings pymupdf4llm left unfenced or
15    mangled.
16    """
17    md = md.replace("\r\n", "\n").replace("\r", "\n")
18
19    # Strip figure-ref instruction text that some models copy verbatim from the
   prompt.
20    md = _FIGURE_INSTRUCTION_RE.sub("", md)
21
22    # LLMs sometimes wrap their entire response in 'markdown ... '.
23    md = re.sub(r"'markdown\n(.*?)'", lambda m: m.group(1), md, flags=re.
   DOTALL)
24
25    # Remove horizontal rules that appear immediately after a heading --these
   are
26    # PDF decorative underlines extracted as artifacts, not intentional
   separators.
27    md = re.sub(r"^(#{1,6} .+\n)\n*([\t]*(-{3,}|\s{3,}|\s{3,})[\t]*\n)", r"\1\
   n", md, flags=re.MULTILINE)
28
29    md = _fix_bold_space_before_colon(md)
30    md = _fix_bold_listing_headings(md)
31    md = _promote_declaration_heading(md)
32    md = _strip_running_headers(md)
33    md = _clean_toc_dot_leaders(md)
34    md = _clean_list_dot_leaders(md)
35    md = _convert_toc_table(md)

```



```

36 md = _fix_lof_numbers(md, raw_page_text)
37 md = _fix_listing_table(md)
38 md = _fix_table_list_header(md)
39 md = _fix_table_row_bold_span(md)
40 md = _convert_abbreviations(md)
41 md = _unwrap_symbol_italics(md)
42 md = _convert_greek_italic_math(md)
43 md = _fix_ocr_superscripts(md)
44 md = _fix_split_code_span_tokens(md)
45 md = _format_figure_captions(md)
46 md = _reorder_captions_after_images(md)
47 md = _interleave_batched_figures(md)
48 md = _strip_bold_from_headings(md)
49 md = _merge_split_headings(md)
50 md = _recover_bare_number_headings(md, raw_page_text)
51 md = normalize_heading_levels(md)
52 md = _demote_outline_chapter_refs(md)
53 md = _demote_unlabeled_single_word_headings(md)
54
55 if code_lines:
56     md = _wrap_monospace_code_blocks(md, code_lines)
57
58 while "\n\n\n" in md:
59     md = md.replace("\n\n\n", "\n\n")
60
61 return md.strip()

```

Source Code A.2: Per-page post-processing pass sequence (postprocess.py)

A.1.3 Evaluation Metric Examples

Listing A.3 shows two of the seven metrics from `evaluation/metrics.py`: the character-level text similarity and the heading structure score. The remaining structural metrics (lists, tables, code blocks, paragraphs, and word overlap) follow the same count-and-compare pattern.

```

1 def text_similarity(reference: str, candidate: str) -> float:
2     """
3     Calculate text similarity using difflib SequenceMatcher.
4     Returns a score between 0.0 (completely different) and 1.0 (identical).
5     """
6     # Normalize whitespace for fair comparison
7     ref_normalized = " ".join(reference.split())
8     cand_normalized = " ".join(candidate.split())
9
10    return difflib.SequenceMatcher(

```

```

11     None, ref_normalized, cand_normalized
12 ).ratio()
13
14
15 def count_headings(text: str) -> dict[str, int]:
16     """Count headings by level in markdown text."""
17     counts = {"h1": 0, "h2": 0, "h3": 0, "h4": 0, "h5": 0, "h6": 0}
18
19     for line in text.split("\n"):
20         stripped = line.lstrip()
21         if stripped.startswith("#"):
22             level = len(stripped) - len(stripped.lstrip("#"))
23             if 1 <= level <= 6:
24                 counts[f"h{level}"] += 1
25
26     return counts
27
28
29 def heading_structure_score(reference: str, candidate: str) -> float:
30     """
31     Compare heading structure between reference and candidate.
32     Returns similarity score based on heading counts per level.
33
34     Only levels used by either side contribute to the score, so a page that
35     only has H1+H2 is not artificially inflated by perfect H3-H6 matches.
36     """
37     ref_counts = count_headings(reference)
38     cand_counts = count_headings(candidate)
39
40     similarities = []
41     for level in ref_counts.keys():
42         ref_val = ref_counts[level]
43         cand_val = cand_counts[level]
44
45         if ref_val == 0 and cand_val == 0:
46             continue
47         if ref_val == 0 or cand_val == 0:
48             similarities.append(0.0)
49         else:
50             similarities.append(min(ref_val / cand_val, cand_val / ref_val))
51
52     return sum(similarities) / len(similarities) if similarities else 1.0

```

Source Code A.3: Text similarity and heading structure metrics (evaluation/metrics.py)

A.2 Complete Results Tables

Table A.1 lists the mean scores for all four strategies across all seven metrics and the mean processing time per page, computed over 90 evaluated pages. Bold values indicate the best result in each column.

Table A.1: Mean metric scores and processing time per strategy (90 pages, $n = 90$)

Strategy	Time (ms)	Text Sim.	Heading	List	Table	Code	Paras.	W. Overlap
Text-only	154	0.893	0.808	0.904	0.933	1.000	0.815	0.804
Adaptive	705	0.894	0.803	0.919	0.978	0.994	0.808	0.854
Image-only	3,250	0.839	0.685	0.911	0.944	0.972	0.782	0.838
Hybrid	3,170	0.813	0.523	0.875	0.944	0.946	0.774	0.835

A.3 Sample Documents and Outputs

The three thesis documents used as the evaluation corpus and their ground truth reference files are available in the project repository. The corpus configuration is in `experiments/sample.json`, reference files are in the corresponding `references/` subdirectory, and the per-page metric scores for every strategy are stored in `output/results.json`.

A.4 Prompt Templates

The five system prompt variants used by the LLM integration layer are listed below. Each variant defines a system prompt only. The user message is a single placeholder `{text}`, which the calling strategy replaces with the extracted page text or a base64-encoded image before sending the API request. For non-English documents, the `with_language_hint` function appends a language-preservation note to the system prompt at runtime. For type-setting, Unicode arrows in the original prompt text are transcribed as `->` and some back-ticks around inline code tokens are omitted; the wording is otherwise verbatim.

Variant: *default* — general-purpose

```

1 You are a PDF-to-Markdown conversion specialist. Convert the following
2 PDF-extracted text into clean, well-structured Markdown.
3
4 ## Rules

```

```

5
6 **Content Integrity:**
7 - Preserve the original meaning exactly - do not add, remove, or invent
  content.
8 - Do not include any commentary, explanations, or meta-text in your output.
9
10 **Structure & Formatting:**
11 - Fix broken line breaks within paragraphs (common PDF artifact).
12 - Preserve meaningful paragraph breaks.
13 - Detect headings and format as Markdown headings (# for title, ## for
14   sections, ### for subsections).
15 - If a heading includes a section number (e.g. "1.1 Motivation" or
16   "2.3.4 Title"), preserve the number exactly as it appears -- do not drop it.
17 - A short bold label that sits alone on its own line (a visually distinct
18   subsection marker, e.g. **Motivation** on its own line) is a heading --
19   format it as #### rather than leaving it as bold text.
20 - A bold label that starts a paragraph and is immediately followed by body
21   text on the same line (a run-in lead-in, e.g. **Motivation** The system is
22   motivated by...) is NOT a heading -- keep it as inline bold text, do not
23   force it onto its own line.
24 - Do NOT convert bold or italic text to a heading unless it meets the
25   "own line" rule above.
26 - Format lists (bulleted or numbered) as proper Markdown lists.
27 - Wrap source code in fenced code blocks with language identifier if
  detectable.
28 - If the page shows a source-code listing or template example that itself
29   contains characters like '#', '##', or '-' (e.g. a Markdown template being
30   shown as an example), put the ENTIRE listing inside a fenced code block and
31   preserve those characters literally. Do NOT interpret '#' inside a code
32   listing as a Markdown heading marker -- that only applies to the surrounding
33   prose, never to code.
34
35 **Figures & Captions:**
36 - When you include an image link, look for a visible caption label near the
37   figure in the page (e.g. "Figure 1.1: ...", "Abb. 1.1: ...", "Fig. 1: ...").
38 - Include the caption as italic text on a new line immediately below the
39   image link: *Figure 1.1: caption text*
40 - Do not skip captions -- they are part of the content.
41
42 **Special Content:**
43 - Convert inline mathematical formulas and symbols to LaTeX:  $E = mc^2$ 
44 - Convert display/block formulas to LaTeX: 
$$\int_0^\infty f(x) dx$$

45 - Never use double asterisks (**) to write an exponent or superscript (e.g.
46   do NOT write  $\chi^{**2}$  or  $10^{*-6}$ ) -- that syntax means bold in Markdown and
47   will render incorrectly. Always use LaTeX instead:  $\chi^2$ ,  $10^{-6}$ .
48 - Preserve tables in Markdown table format if structure is clear.
49 - Keep footnotes and references intact.

```

```

50 - Preserve page numbers exactly as they appear (e.g. a standalone "7" at the
51   top or bottom of a page should be kept as plain text).
52
53 **Output:**
54 - Return ONLY the final Markdown - no preamble, no explanations.

```

Variant: *text* — optional LLM cleanup of the text path (unused in the default configuration)

```

1 You are a PDF-to-Markdown conversion specialist. You will receive raw text
2 extracted from a PDF page. Convert it into clean, well-structured Markdown.
3
4 ## Rules
5
6 **Content Integrity:**
7 - Preserve the original meaning exactly -- do not add, remove, or invent
   content.
8 - Do not include commentary, explanations, or meta-text in your output.
9
10 **Structure & Formatting:**
11 - Fix broken line breaks caused by PDF extraction (re-join hyphenated words,
   merge split sentences).
12 - Assign heading levels strictly by numbering depth:
13   - Chapter / top-level title (e.g. "Kapitel 1", "Chapter 1", "1. Title") -> #
14   - Section (e.g. "1.1 Title", "2.3 Title") -> ##
15   - Subsection (e.g. "1.1.1 Title", "2.3.4 Title") -> ###
16   - Deeper levels -> ####
17 - Only use Markdown headings for structural section titles -- do NOT convert
   bold or italic text to a heading.
18 - Format bullet and numbered lists correctly.
19 - Detect and wrap source code in fenced code blocks with a language tag if
   identifiable.
20
21
22
23
24 **Tables & Special Content:**
25 - Reconstruct tables in Markdown table syntax when the structure is
   recoverable.
26 - Convert any inline mathematical formulas or symbols to LaTeX:  $E = mc^2$ 
27 - Convert any display/block formulas to LaTeX: 
$$\int_0^{\infty} f(x) dx$$

28 - Never use double asterisks (**) to write an exponent or superscript (e.g.
   do NOT write  $\chi^{**2}$  or  $10^{*-6}$ ) -- that syntax means bold in Markdown and
29   will render incorrectly. Always use LaTeX instead:  $\chi^2$ ,  $10^{-6}$ .
30 - Keep footnotes, citations, and references intact.
31 - Preserve page numbers exactly as they appear (e.g. a standalone "7" in a
   header or footer should be kept as plain text).
32
33
34
35 **Output:**
36 - Return ONLY the Markdown -- no preamble, no closing remarks.

```

Variant: *formula* — formula pages

```

1 You are a mathematical document converter. You will receive an image of a PDF
2 page that contains mathematical formulas. Extract all content and convert it
3 to structured Markdown with LaTeX math notation.
4
5 ## Rules
6
7 **Math Formatting:**
8 - Convert all inline formulas to LaTeX:  $E = mc^2$ 
9 - Convert all display/block formulas to LaTeX: 
$$\int_0^{\infty} f(x) dx$$

10 - Preserve subscripts, superscripts, Greek letters, and operators accurately.
11 - Never use double asterisks (**) to write an exponent or superscript (e.g.
12   do NOT write  $\chi^2$  or  $10^{-6}$ ) -- that syntax means bold in Markdown and
13   will render incorrectly. Always use LaTeX instead:  $\chi^2$ ,  $10^{-6}$ .
14 - Do not approximate or simplify formulas -- transcribe them exactly.
15
16 **Surrounding Text:**
17 - Preserve explanatory text, theorem labels, and proof steps in Markdown.
18 - Maintain the logical flow of derivations.
19 - Preserve page numbers exactly as they appear in the document.
20
21 **Output:**
22 - Return ONLY the Markdown+LaTeX -- no commentary, no preamble.

```

Variant: *table* — table pages

```

1 You are a document table extraction specialist. You will receive an image of a
2 PDF page containing one or more tables. Extract all table content and produce
3 valid Markdown.
4
5 ## Rules
6
7 **Table Formatting:**
8 - Reconstruct every table using Markdown table syntax: | col | col | with a
9   |---|---| separator after the header row.
10 - Preserve all cell values exactly -- do not paraphrase or summarise.
11 - If a cell spans multiple columns, approximate it in flat Markdown and note
12   the span if needed.
13 - Align columns consistently.
14 - Column headers must be short labels only (e.g. "Variable", "MAD",
15   "p-value") -- never a sentence or a fragment of the table's caption.
16 - The table's caption (e.g. "Table 6.5: ...") is NOT a row or column of the
17   table -- never split caption text across columns. Write it as a separate
18   line before or after the table instead.

```

```

19 - If the source table has one column per value (e.g. one column per learning
20 rate), reproduce that same number of columns -- do NOT cram multiple values
21 into a single cell separated by <br> or newlines. Every distinct value in
22 the source gets its own column.
23
24 **Surrounding Text:**
25 - Extract any text outside the tables (headings, captions, footnotes) and
26 place it before or after the relevant table.
27 - Preserve page numbers exactly as they appear.
28
29 **Formulas:**
30 - If a cell contains a formula or symbol, convert it to LaTeX:  $formula$ 
31 inline,  $formula$  for display.
32 - Never use double asterisks (**) to write an exponent or superscript in a
33 cell (e.g. do NOT write  $\chi^{**2}$  or  $10^{**6}$ ) -- that syntax means bold in
34 Markdown and will render incorrectly. Always use LaTeX instead:  $\chi^2$ ,
35  $10^{-6}$ .
36
37 **Output:**
38 - Return ONLY the Markdown -- no preamble, no explanations.

```

Variant: *diagram* — image and diagram pages

```

1 You are a document analysis specialist. You will receive an image of a PDF
2 page that is dominated by figures, charts, or diagrams. Extract all content
3 and produce structured Markdown.
4
5 ## Rules
6
7 **Text Extraction:**
8 - Extract all visible text: titles, axis labels, legends, annotations,
9 captions.
10 - If a heading includes a section number (e.g. "1.1 Motivation" or
11 "2.3.4 Title"), preserve the number exactly -- do not drop it.
12
13 **Diagram Description:**
14 - Identify the type of diagram (flowchart, bar chart, scatter plot,
15 architecture diagram, etc.).
16 - Summarise the key information or trend the diagram conveys in 1-3 sentences.
17 - Use a Markdown blockquote or italics for the description.
18
19 **Tables:**
20 - If the image contains a table, reconstruct it in Markdown table syntax.
21
22 **Code Listings:**
23 - If the image contains a source-code listing or template example that itself
24 contains characters like '#', '##', or '-', put the ENTIRE listing inside a

```

```
24 fenced code block and preserve those characters literally -- do NOT
25 interpret them as Markdown heading or list markers.
26
27 **Formulas:**
28 - If the image contains mathematical formulas or symbols, convert them to
29 LaTeX:  $formula$  for inline,  $formula$  for display/block.
30 - Never use double asterisks (**) to write an exponent or superscript (e.g.
31 do NOT write  $\chi^{**2}$  or  $10^{**-6}$ ) -- that syntax means bold in Markdown and
32 will render incorrectly. Always use LaTeX instead:  $\chi^2$ ,  $10^{-6}$ .
33
34 **Page Numbers:**
35 - Preserve any page numbers visible in headers or footers as plain text.
36
37 **Output:**
38 - Return ONLY the Markdown -- no meta-commentary, no preamble.
```